

Gebo.ai configuration manual (1.0.2.2)

Enterprise open source retrieve augmented generation, chatbot and agents platform.

Sommario

Gebo.ai “monolithic version” on premise installation structure	5
Required software for the installation	5
Esyinstall docker installation	6
Docker compose installation.....	6
Preparation from scratch of an Ubuntu server 24.04 system to run the docker-compose distribution of Gebo.ai	7
Mounting windows shared filesystems (SMB CIFS) to be visible to your gebo.ai linux server: ..	8
Installing Gebo.ai from a preconfigured appliance	8
Installing Gebo.ai on premise or in your private/public cloud	9
Installation steps on your on custom installation:.....	9
After software installation step	9
Linux Ubuntu and “Debian style” installation:	10
Prerequisites.....	10
Install mongodb community edition.....	11
Install QDRANT vector database	14
Install the Neo4J graph database (community).....	17
Install OpenSearch full text search engine/server.....	19
Gebo.ai software installation.....	19
Enabling the firewall to let you reach the gebo.ai web port:	21
Ensure your server time is ok.	21

Ensure all services are booted at system startup.....	22
Gebo.ai on premise linux standard paths and infos:.....	22
Upgrade Ubuntu gebo.ai installation	22
Data paths (to be backedup properly).....	23
After software installation step	23
Installing Gebo.ai on a windows server	23
Hybrid docker compose/windows installation	23
Installing MongoDB/Qdrant/Neo4J/Gebo.ai on the windows operating system.	26
“Monolithic version” base installation components & integration with 3 rd party systems.	27
Minimum software stack for the installation.....	27
Connectivity with cloud generative AI providers or local generative AI server	27
Supported providers (and some supported models examples).....	27
Connectivity with company used on premise or cloud systems	28
2 nd composing knowledge bases.....	28
Gebo.ai detailed configuration	29
GEBO_HOME environment variable or system property	29
GEBO_WORK_DIRECTORY environment variable or system property or interactive admin UI setting.....	29
MongoDB connectivity settings	30
QDRANT or other vector database configuration	30
Neo4J database configuration.....	31
OpenSearch database configuration	31
Company shared filesystem	32
Gebo.ai — User & Admin screens Software Guide	32
First-run setup.....	32
Create your local admin account	32
The setup checklist — your map of the whole application.....	33
Connecting your first LLM provider.....	33
Easy initial setup — pick a vendor, paste a key	33
Extended infrastructure/suppliers choices	34
Result: six model types provisioned from one API key.....	35
After: chat is live	36
The full setup checklist	37

Administrator & others account setup	37
Single sign-on, OAuth2 authentication	38
Configuration purpose — the same for every provider.....	39
Atlassian Confluence integration	43
Atlassian Jira integration	43
Microsoft SharePoint integration	44
AWS S3 integration	45
Google Drive/Workspaces integration	46
Configure at least a knowledge base	48
Configure Google search API	50
Configure MCP Servers	51
Configure Exposed MCP Servers.....	53
Configure Administrative API Keys	55
Integrations, API keys & your profile	56
Api Keys/MCP — Model Context Protocol servers.....	56
Edit profile	57
End-to-end: upload a document & RAG over it	58
Step 1 — Create the knowledge base	58
The working path — Admin's knowledge base tree.....	60
Gebo.ai hardening guide, architecture overview	67
Monolithic (`gebo.ai.app`)	67
Microservices (19 distinct services).....	67
Shared Infrastructure (both deployments)	67
2. Part A — Enabling HTTPS (SSL/TLS).....	68
2.1 Monolithic Deployment	68
Step 1 — Generate a PKCS12 Keystore	68
Step 2 — Mount the Keystore in Docker	69
Step 5 — Update CORS Configuration	70
Step 6 — Restart.....	70
2.2 Microservices Deployment	70
2.3 Certificate Generation Reference	72
3. Part B — Externalising the Encryption Keystore.....	73

3.3 Configuring the External Keystore — Monolith	74
4. Part C — Secrets Rotation Checklist	76
JWT Secret Rotation Procedure.....	77
Environment Variable Injection (Recommended)	77
5. Part D — Network & Infrastructure Hardening	78
5.1 Docker Network Binding	78
5.2 Microservices — Cluster Participant Restriction	78
5.3 OpenSearch — Internal SSL	78
5.4 Qdrant TLS.....	79
5.5 Firewall Rules	79

Gebo.ai “monolithic version” on premise installation structure

This document lists both already developed and under development integrations.

The “monolithic version” is called so contraposed to the “Microservices version” that will be available soon, even if suboptimal “Monolithic” software in this case is easier to install and already tested for installations with up to 10000 documents indexed and retrieved in the chat context.

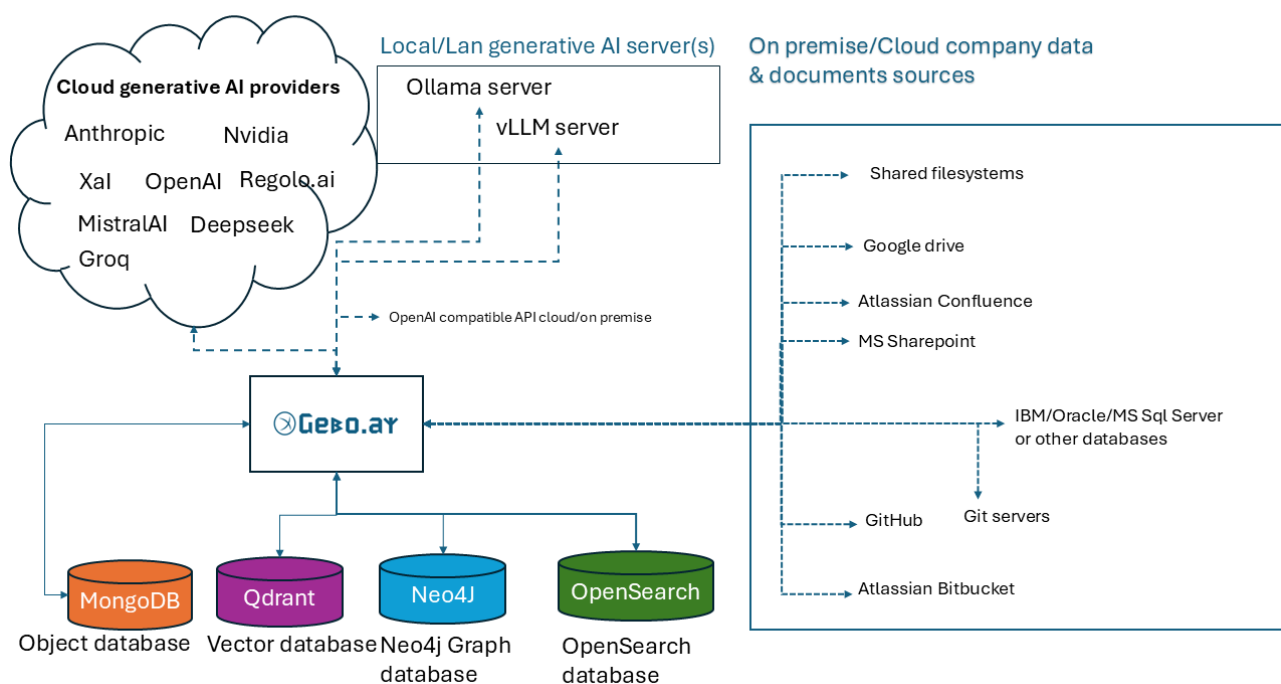


Figure 1 Gebo.ai on premise installation components and 3rd party integration schema

Required software for the installation

All the required software is installed automatically in both the choice to use an already configured appliance (virtual machine) taken from the download page of our site or by using the latest docker-compose.yml docker file.

The on premise installation requires:

- A java virtual machine from 21+
- MongoDB community or other version
- QDrant vector database
- Optional: Neo4J graph database (optional)
- OpenSearch full text retrieval database (optional but highly suggested).
- Connectivity to the chosen AI provider or local infrastructure.

Esyinstall docker installation

Image name: geboai/easyinstall.gebo.ai

Public docker repository: <https://hub.docker.com/r/geboai/easyinstall.gebo.ai>

This is the “go to” installation for initial tests, will not enable graphrag feature but you can use all the retrieve augmented generation features, erogating chatbots with a fully featured multilanguage/multiuser application from a single docker image.

You can simply launch:

docker run -p 12999:12999 --name local-gebo-ai geboai/easyinstall.gebo.ai:latest

After few seconds you can connect to: <http://<your server>:12999/> ad start configuring your software.

With this image you can scale to a full onpremise installation by extracting all its components backups (gebo.ai software, mongodb backup, qdrant backup) and restoring them in your custom installation.

Important prerequisites: **if you plan to use the easyinstall installation on a linux host operating system, ensure to run a “UTF-8” locale like en_US.UTF-8 localization or it_IT.UTF-8 (for Italian) this is mandatorly if you plan to mount windows filesystem shares to be accessible to Gebo.ai.**

Docker compose installation

Image name: geboai/gebo.ai

Public docker repository: <https://hub.docker.com/r/geboai/gebo.ai>

Important prerequisites: **if you plan to use the docker-compose installation on a linux host operating system, ensure to run a “UTF-8” locale like en_US.UTF-8 localization or it_IT.UTF-8 (for Italian) this is mandatory if you plan to mount windows filesystem shares to be accessible to Gebo.ai.**

On ubuntu server for example run:

```
#sudo apt-get install language-pack-en
```

```
#sudo locale-gen en_US.UTF-8
```

```
#update-locale LANG=en_US.UTF-8
```

The provided docker compose

(<https://raw.githubusercontent.com/geboai/Gebo.ai/refs/heads/develop/dockers/gebo.ai/windows/docker-compose.yml>) **docker-compose.yml** configuration files works without any required modification.

It setups both MongoDB, Qdrant, Gebo.ai setting up their connectivity and exporting the port 12999 where the gebo.ai application is bound.

The docker-compose configuration let the 3 applications run as a service, so if in your system dockerd or the containerized subsystem is running as a service with automatic startup at system boot, you are fully ready on erogating this service on your local network.

It is very productive to create an “up and running” quite performant installation but contains security settings like mongo password and qdrant api key that we suggest to change for production use (go to the security considerations section to check these topics).

1st Starting up the application

The fastest way to obtain an up & running environment is copying the provided docker-compose.yml in a folder and run:

user@localhost#docker-compose up -d

2nd point your browser to the <server address>:12999 port

This will start the application setup in your browser, the absolute mandatory step here is to register an administrator user with a password.

All others configuration steps now are possible with a system that will be already having a restrict access to an administrator user that will be able to configure new users and all the other technical details of the system.

Preparation from scratch of an Ubuntu server 24.04 system to run the docker-compose distribution of Gebo.ai

If you require to prepare a system from scratch to use Gebo.ai follow the instructions:

- Install Ubuntu server 24.04 LTS
- Adjust the character encoding of your system
 - o Install language pack with:”**sudo apt-get install language-pack-en**” (or other language pack of your local language or company filesystems locales)
 - o Run “**locale-gen en_US.UTF-8**”
 - o Run “**sudo update-locale LANG=en_US.UTF-8**”
- Install and configure docker
 - o Run “**sudo apt-get install docker.io**”
 - o Run “**sudo apt-get install docker-compose**”
 - o Run “**sudo service docker start**”

- Create gebo.ai user to run docker-compose system:
 - o check docker group gid in /etc/group (110)
 - o create gebo.ai user with group docker and bourn shell as a shell, home directory /home/gebo.ai with: “**sudo useradd gebo.ai -g 110 -m -s /bin/bash**”
 - o create a gebo.ai use password with: “**sudo passwd gebo.ai**”
- Launching gebo.ai infrastructure
 - o Download docker-compose.yml from <https://gebo.ai/> in /home/gebo.ai
 - o Switch user to gebo.ai “**sudo su – gebo.ai**”
 - o Launching the infrastructure: “**sudo docker-compose up -d**”

Mounting windows shared filesystems (SMB CIFS) to be visible to your gebo.ai linux server:

If you plan to host gebo.ai in a linux server or in a linux server using docker-compose (see preceeding chapters), please ensurie that your linux service ha an extensive support for eventually present UTF-8 characters encoding in file names (view preceeding chapters).

To successfully let your windows shared filesystem being visible to your gebo.ai server let’s mount them with the following commands:

```
#mount //<server name>/<share path> /mnt/windows-share -t cifs -o
username=<username>,password=<password>,uid=<uid nr>,gid=<gid nr>,iocharset=utf8
```

Or

```
#mount //<server name>/<share path> /mnt/windows-share -t cifs -o
username=<username>,password=<password>,uid=<uid nr>,gid=<gid
nr>,iocharset=utf8,codepage=cp850
```

You can statically provide the reference to your company’s windows shares to your /etc/fstab configuration file and configure those mounted volumes to be visible to your gebo.ai system.

Installing Gebo.ai from a preconfigured appliance

You can download a preconfigured virtual machine with all required software already installed

Download the appliance from: <https://gebo.ai/downloads/>

On the appliance accounts are predetermined, please change them to harden your installation.

Accounts informations are in `accounts.txt` file contained in downloadable appliances archives.

Installing Gebo.ai on premise or in your private/public cloud

This is the “go to” installation for companies who want to take the most out of being fully configurable with custom security settings.

Check on: <https://gebo.ai/downloads/> latest Gebo.ai installation image for your system.

GitHub releases are on: <https://github.com/geboai/Gebo.ai/releases/>

Installation steps on your on custom installation:

Prerequisites: if you use a linux operating system ensure to install full UTF-8 support for your filesystems, otherwise accessing company shared Unicode filesystems will be nearly impossible.

1st install mongodb community edition <https://www.mongodb.com/try/download/community>

2nd install QDRANT <https://qdrant.tech/documentation/guides/installation/#installation-options>

3rd install NEO4J Graph database, is optional, you can disable it, but it enables Graphrag features with unprecedented documents retrieval performance. It is experimental in the version 1.0.0.0-rc1 but will be improved on following versions.

4rd install Gebo.ai from a binary distribution (or directly the spring boot bootable jar).

After software installation step

Go with your browser to: `http:<your server address>:12999/` create your administrator account and start using/administering the software for all the company.

Linux Ubuntu and “Debian style” installation:

We’ll use ubuntu server 24.04.3, after the operating system installation, log in your server:

```
zava@gebo-ai-server: ~  
Esecuzione di Ping gebo-ai-server [192.168.148.142] con 32 byte di dati:  
Risposta da 192.168.148.142: byte=32 durata<1ms TTL=64  
Risposta da 192.168.148.142: byte=32 durata<1ms TTL=64  
Risposta da 192.168.148.142: byte=32 durata<1ms TTL=64  
Risposta da 192.168.148.142: byte=32 durata<1ms TTL=64  
Statistiche Ping per 192.168.148.142:  
  Pacchetti: Trasmessi = 4, Ricevuti = 4,  
  Persi = 0 (0% persi),  
  Tempo approssimativo percorsi andata/ritorno in millisecondi:  
    Minimo = 0ms, Massimo = 1ms, Medio = 0ms  
  
Paolo@ZBOOKSTATION MINGW64 ~  
$ ssh zava@gebo-ai-server  
The authenticity of host 'gebo-ai-server (192.168.148.142)' can't be established.  
ED25519 key fingerprint is SHA256:kHQ0mkLyHsYKqmoCvxZd0n3uyZAZz1kyFIT9FB/w1Ds.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'gebo-ai-server' (ED25519) to the list of known hosts.  
zava@gebo-ai-server's password:  
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-88-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
System information as of Fri Dec 5 07:04:08 AM UTC 2025  
  
System load:  0.0          Processes:      248  
Usage of /:   30.9% of 23.45GB  Users logged in: 1  
Memory usage: 4%          IPv4 address for ens33: 192.168.148.142  
Swap usage:  0%  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
zava@gebo-ai-server:~$ |
```

Prerequisites:

Adjust the character encoding of your system

- Install language pack with: “**sudo apt-get install language-pack-en**” (or other language pack of your local language or company filesystems locales)
- Run “**locale-gen en_US.UTF-8**”
- Run “**sudo update-locale LANG=en_US.UTF-8**”

Install a java jdk version 21 or upper:

Run:

```
# sudo apt install java-common
```

Install java jdk 21 (for qdrant or to run gebo.ai directly from spring boot jar)

```
# sudo apt install openjdk-21-jre-headless
```

To check your installation run:

```
#java -version
```

The correct answer is similar to

```
openjdk version "21.0.9" 2025-10-21
```

Run commands (responding Y as yes to interactive questions):

```
# sudo apt-get install language-pack-en
```

```
#sudo locale-gen en_US.UTF-8
```

```
# sudo update-locale LANG=en_US.UTF-8
```

This will enable general UTF-8 support, select your custom localization here will improve the adaptation to your Unicode company systems.

Install mongodb community edition

Go to: <https://www.mongodb.com/try/download/community>

Select your operating system.

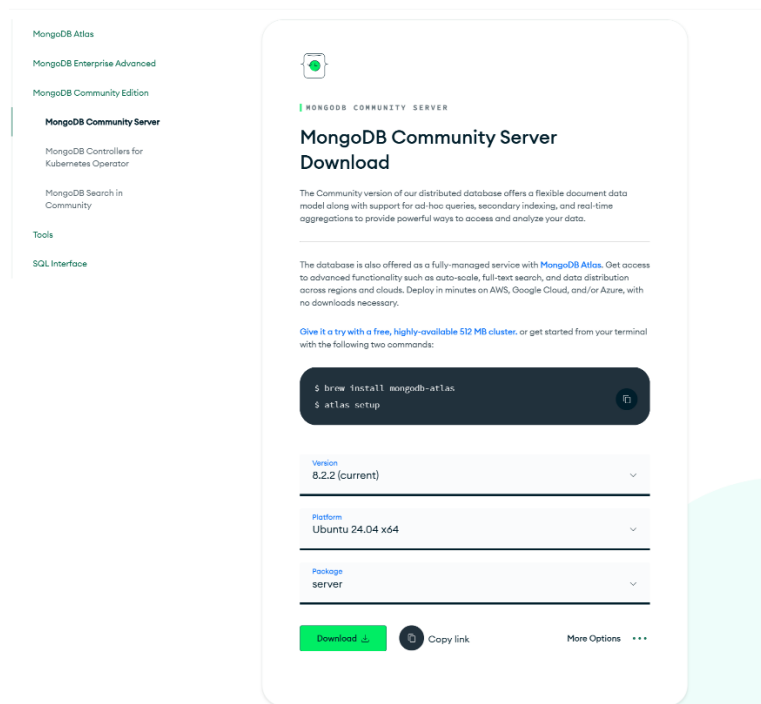


Figure 2 mongodb community download user interface

You can download directly in your server by running:

```
#wget https://repo.mongodb.org/apt/ubuntu/dists/noble/mongodb-org/8.2/multiverse/binary-amd64/mongodb-org-server_8.2.2_amd64.deb
```

After the download install mongodb running:

```
# sudo apt install ./mongodb-org-server_8.2.2_amd64.deb
```

Let the service start with:

```
# service mongod start
```

Now install utils and mongo administrative tools:

```
# sudo apt-get update
```

```
# sudo apt-get install -y gnupg curl
```

Install the mongo repository key and repository references

```
#curl -fsSL https://www.mongodb.org/static/pgp/server-8.0.asc \
```

```
| sudo gpg -o /usr/share/keyrings/mongodb-server-8.0.gpg --dearmor
```

```
#echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ] https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 multiverse" \
```

```
| sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list
```

```
#sudo apt-get update
```

```
#sudo apt install mongodb-mongosh
```

Now create a mongo general administrator with its username and password

Access the mongosh shell:

```
#mongosh "mongodb://localhost:27017/admin"
```

Now on the mongosh shell:

Superuser creation (restricts the full administrative features access)

```
> use admin
```

```
admin> db.createUser({
```

```
  user: "mongoadmin",
```

```
  pwd: "<put here your password>",
```

```
  roles: [
```

```
    { role: "root", db: "admin" }
```

```
  ]
```

```
})
```

Create a restricted user capable of accessing/creating databases

```
admin> db.createUser({
```

```
  user: "gebo-ai",
```

pwd: ""<put here your password>",

roles: [

{ role: "readWriteAnyDatabase", db: "admin" },

{ role: "dbAdminAnyDatabase", db: "admin" },

{ role: "userAdminAnyDatabase", db: "admin" }

]

})

Quit mongosh

admin>quit

Startup the mongod service

#service mongod start

Check the mongod service with:

#service mongod status

The correct status is this:

```
root@gebo-ai-server: ~
root@gebo-ai-server:~# service mongod status
● mongod.service - MongoDB Database Server
   Loaded: loaded (/usr/lib/systemd/system/mongod.service; disabled; preset: enabled)
   Active: active (running) since Fri 2025-12-05 09:41:59 UTC; 30s ago
     Docs: https://docs.mongodb.org/manual
    Main PID: 6106 (mongod)
  Memory: 323.0M (peak: 402.2M)
    CPU: 475ms
  CGroup: /system.slice/mongod.service
          └─6106 /usr/bin/mongod --config /etc/mongod.conf

Dec 05 09:41:59 gebo-ai-server systemd[1]: Started mongod.service - MongoDB Database Server.
Dec 05 09:41:59 gebo-ai-server mongod[6106]: {"t":{"$date":"2025-12-05T09:41:59.846+00:00"},"s":"I",  "c":"-",      "id":8991200, "ctx":"main","msg":"Sh
Dec 05 09:41:59 gebo-ai-server mongod[6106]: {"t":{"$date":"2025-12-05T09:41:59.852+00:00"},"s":"I",  "c":"CONTROL",  "id":7484500, "ctx":"main","msg":"En
lines 1-13/13 (END)
```

Install QDRANT vector database

Get latest release from this base url: <https://github.com/qdrant/qdrant/releases>

Download binaries directly from **GitHub**, run:

```
#wget https://github.com/qdrant/qdrant/releases/download/v1.16.2/qdrant_1.16.2-1_amd64.deb
```

Install with:

```
#sudo apt install ./qdrant_1.16.2-1_amd64.deb
```

If this release does not install a proper systemd compliant service, create a qdrant user

```
#sudo useradd --system --no-create-home --shell /usr/sbin/nologin qdrant
```

Check that standard folders exists

```
#sudo mkdir -p /var/lib/qdrant/storage \
               /var/lib/qdrant/snapshots \
               /var/lib/qdrant/static \
               /etc/qdrant
```

Change ownership of all folders to qdrant user

```
#sudo chown -R qdrant:qdrant /var/lib/qdrant /etc/qdrant
```

Check if standard qdrant configuration file is in place, provide a configuration file with a proper api key to protect your accesses:

```
# sudo nano /etc/qdrant/config.yaml
```

Provide the following entries to the .yaml file (pay attention to yaml formatting standard)

```
log_level: INFO
storage:
  storage_path: "/var/lib/qdrant/storage"
  snapshot_path: "/var/lib/qdrant/snapshots"
server:
  http_port: 6333
  grpc_port: 6334
# basic API key protection
service:
  api_key: "<enter here a uuid api key>"
```

now we create the systemd service:

sudo nano /etc/systemd/system/qdrant.service

Copy this code inside

```
[Unit]
Description=Qdrant Vector Database
After=network.target

[Service]
User=qdrant
Group=qdrant
WorkingDirectory=/var/lib/qdrant
ExecStart=/usr/bin/qdrant --config-path /etc/qdrant/config.yaml
Restart=on-failure
# Allow Qdrant to keep many files open (good for bigger workloads)
LimitNOFILE=10000

[Install]
WantedBy=multi-user.target
```

Save the code and enable the qdrant service running:

#sudo systemctl daemon-reload

#sudo systemctl enable --now qdrant

Check the service status:

#systemctl status qdrant

This is a correct status screen (if you don't see this try restarting the qdrant service):

```
zava@gebo-ai-server: ~  
zava@gebo-ai-server:~$ systemctl status qdrant  
● qdrant.service - Qdrant Vector Database  
   Loaded: loaded (/etc/systemd/system/qdrant.service; enabled; preset: enabled)  
   Active: active (running) since Fri 2025-12-05 08:35:46 UTC; 2min 36s ago  
 Main PID: 1690 (qdrant)  
    Tasks: 22 (limit: 9376)  
  Memory: 55.5M (peak: 57.8M)  
     CPU: 173ms  
    CGroup: /system.slice/qdrant.service  
            └─1690 /usr/bin/qdrant --config-path /etc/qdrant/config.yaml  
  
Dec 05 08:35:46 gebo-ai-server qdrant[1690]: 2025-12-05T08:35:46.890797Z INFO storage::content_manager::consensus::persistent: Loading raft state from /v  
Dec 05 08:35:46 gebo-ai-server qdrant[1690]: 2025-12-05T08:35:46.894614Z INFO qdrant: Distributed mode disabled  
Dec 05 08:35:46 gebo-ai-server qdrant[1690]: 2025-12-05T08:35:46.894816Z INFO qdrant: Telemetry reporting enabled, id: 8cdf035-06f3-439c-9f2d-34112a22ce  
Dec 05 08:35:46 gebo-ai-server qdrant[1690]: 2025-12-05T08:35:46.906417Z INFO qdrant::actix: TLS disabled for REST API  
Dec 05 08:35:46 gebo-ai-server qdrant[1690]: 2025-12-05T08:35:46.906795Z INFO qdrant::actix: Qdrant HTTP listening on 6333  
Dec 05 08:35:46 gebo-ai-server qdrant[1690]: 2025-12-05T08:35:46.906997Z INFO actix_server::builder: starting 3 workers  
Dec 05 08:35:46 gebo-ai-server qdrant[1690]: 2025-12-05T08:35:46.907006Z INFO actix_server::server: Actix runtime found; starting in Actix runtime  
Dec 05 08:35:46 gebo-ai-server qdrant[1690]: 2025-12-05T08:35:46.907010Z INFO actix_server::server: starting service: "actix-web-service-0.0.0:6333", w  
Dec 05 08:35:46 gebo-ai-server qdrant[1690]: 2025-12-05T08:35:46.914011Z INFO qdrant::tonic: Qdrant gRPC listening on 6334  
Dec 05 08:35:46 gebo-ai-server qdrant[1690]: 2025-12-05T08:35:46.914044Z INFO qdrant::tonic: TLS disabled for gRPC API  
lines 1-20/20 (END)
```

Check local connectivity:

#curl <http://localhost:6333/>

A correct response looks like this:

```
zava@gebo-ai-server: ~  
zava@gebo-ai-server:~$ curl http://localhost:6333/  
{  
  "title": "qdrant - vector search engine",  
  "version": "1.16.2",  
  "commit": "d2834de0b51be23a7b22b023e424cbb9456d0e75"  
}
```

Refer to: <https://qdrant.tech/documentation/guides/installation/>

```

root@gebo-ai-server: ~
● neo4j.service - Neo4j Graph Database
   Loaded: loaded (/usr/lib/systemd/system/neo4j.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-12-05 09:10:07 UTC; 36s ago
 Main PID: 4338 (java)
    Tasks: 89 (limit: 9376)
  Memory: 505.3M (peak: 544.0M)
     CPU: 8.867s
    CGroup: /system.slice/neo4j.service
            └─4338 /usr/bin/java -Xmx128m -classpath "/usr/share/neo4j/lib/*:/usr/share/neo4j/etc:/usr/share/neo4j/repo/*" -Dapp.name=neo4j -Dapp.pid=4338
              └─4369 /usr/lib/jvm/java-21-openjdk-amd64/bin/java -cp "/var/lib/neo4j/plugins/*:/etc/neo4j/*:/usr/share/neo4j/lib/*" -XX:+UseG1GC -XX:-OmitSp

Dec 05 09:10:09 gebo-ai-server neo4j[4369]: 2025-12-05 09:10:09.580+0000 INFO This instance is ServerId{e411a34a} (e411a34a-fd01-41e0-9767-eb6e5e9c0785)
Dec 05 09:10:10 gebo-ai-server neo4j[4369]: 2025-12-05 09:10:10.157+0000 INFO ===== Neo4j 5.26.18 =====
Dec 05 09:10:10 gebo-ai-server neo4j[4369]: 2025-12-05 09:10:11.355+0000 INFO Anonymous Usage Data is being sent to Neo4j, see https://neo4j.com/docs/usage
Dec 05 09:10:11 gebo-ai-server neo4j[4369]: 2025-12-05 09:10:11.374+0000 INFO Bolt enabled on localhost:7687.
Dec 05 09:10:11 gebo-ai-server neo4j[4369]: 2025-12-05 09:10:11.771+0000 INFO HTTP enabled on localhost:7474.
Dec 05 09:10:11 gebo-ai-server neo4j[4369]: 2025-12-05 09:10:11.771+0000 INFO Remote interface available at http://localhost:7474/
Dec 05 09:10:11 gebo-ai-server neo4j[4369]: 2025-12-05 09:10:11.773+0000 INFO id: 3A622222835F4CB220EB3FE6A525C12D81DAC4CF6042E2D19125D4997918B9AA
Dec 05 09:10:11 gebo-ai-server neo4j[4369]: 2025-12-05 09:10:11.773+0000 INFO name: system
Dec 05 09:10:11 gebo-ai-server neo4j[4369]: 2025-12-05 09:10:11.773+0000 INFO creationDate: 2025-12-05T09:10:10.807Z
Dec 05 09:10:11 gebo-ai-server neo4j[4369]: 2025-12-05 09:10:11.773+0000 INFO Started.

lines 1-21/21 (END)

```

Now neo4j is set with user: neo4j and password: neo4j, change the password with:

#cypher-shell -u neo4j -p neo4j

The software will ask you a new password.

If the system does not ask you the password let's run :

Neo4j@neo4j> **ALTER CURRENT USER SET PASSWORD FROM 'neo4j' TO '<neo4j
password>;**

Install OpenSearch full text search engine/server

Download the OpenSearch application for your platform here:

<https://opensearch.org/downloads/>

Installing instructions: <https://docs.opensearch.org/latest/install-and-configure/install-opensearch/debian/>

Execute:

Installation:

```
#env OPENSEARCH_INITIAL_ADMIN_PASSWORD=<your opensearch password> dpkg -i  
opensearch-3.6.0-linux-x64.deb
```

Enablement of the service:

```
#systemctl enable opensearch
```

Starting up the service:

```
#systemctl start opensearch
```

Check the installation with:

```
# systemctl status opensearch
```

Gebo.ai software installation

Download actual gebo-ai version from the <https://gebo.ai/downloads/> or from <https://github.com/geboai/Gebo.ai/releases/> web page (select .deb format).

Run:

```
#sudo apt install ./gebo-ai_1.0.0.0-rc1_amd64.deb
```

Now edit the configuration file for the gebo.ai application reporting there all your connectivity

Informations for mongo, qdrant and neo4j:

```
#sudo nano /etc/gebo-ai/application.properties
```

Now edit the file:

```
logging.level.root=INFO  
logging.level.org.springframework=INFO  
logging.level.org.springframework.web=INFO  
logging.level.org.springframework.core=INFO  
logging.level.org.springframework.core.codec=INFO
```

```
server.port=12999
server.compression.enabled=true
server.compression.mime-
types=text/html,text/xml,text/plain,text/css,text/javascript,application/javascript,application/json
server.compression.min-response-size=1024
server.http2.enabled=true
server.servlet.contextPath=/
spring.servlet.multipart.enabled=true
spring.servlet.multipart.maxFileSize=100MB
spring.servlet.multipart.maxRequestSize=100MB

#Neo4J connectivity configuration
ai.gebo.neo4j.enabled=true
spring.neo4j.uri=bolt://localhost:7687
spring.neo4j.authentication.username=neo4j
spring.neo4j.authentication.password=<write here your neo4j password>

#Eventual custom token secret for unique JWT in installation
#ai.gebo.security.auth.tokenSecret=04ca023b3...
#ai.gebo.security.auth.tokenExpirationMsec=120000
#ai.gebo.security.cors.allowedOrigins=http://localhost:12999,http://localhost:4200

#Qdrant connectivity configuration
ai.gebo.vectorstore.use=QDRANT
ai.gebo.vectorstore.qdrant.host=127.0.0.1
ai.gebo.vectorstore.qdrant.port=6334
ai.gebo.vectorstore.qdrant.tls=false
ai.gebo.vectorstore.qdrant.apiKey=<write here your qdrant api key>
#Work directory can be configured using UI
ai.gebo.config.setupConfiguresWorkdir=false

#MongoDB connectivity data
ai.gebo.mongodb.enabled=true
ai.gebo.mongodb.databaseName=gebo-ai
ai.gebo.mongodb.connectionString=mongodb://gebo-ai:<wite the mongo gebo-ai
password >@localhost:27017/gebo-ai?authSource=admin

#OpenSearch connectivity data
ai.gebo.opensearch.enabled=true
ai.gebo.opensearch.protocol=https
```

```
ai.gebo.opensearch.host=127.0.0.1
ai.gebo.opensearch.port=9200
ai.gebo.opensearch.username=admin
ai.gebo.opensearch.password=<your opensearch admin password>

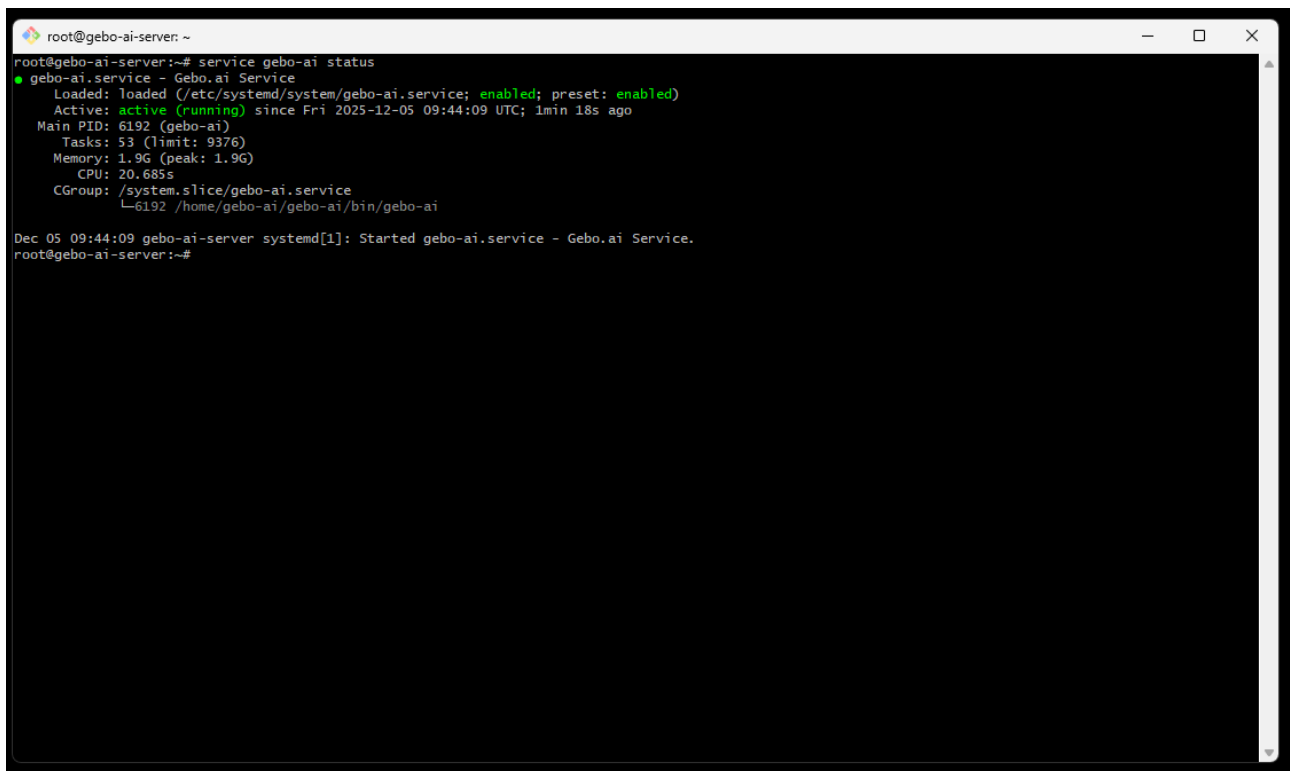
#Allow user connecting configuring wick share(s) to configure from UI
ai.gebo.filesystem.allowFilesystemSharesUI=true
ai.gebo.filesystem.shares=
```

Startup the gebo-ai service:

```
#sudo service gebo-ai start
```

Check the gebo-ai service with:

```
#sudo service gebo-ai status
```

A terminal window titled 'root@gebo-ai-server: ~' showing the output of the command 'service gebo-ai status'. The output indicates that the 'gebo-ai.service' is loaded, active, and running. It provides details such as the main PID (6192), tasks (53), memory usage (1.9G), CPU usage (20.685s), and the CGroup path. At the bottom, it shows a log message from systemd stating that the service was started on Dec 05 09:44:09.

```
root@gebo-ai-server: ~
root@gebo-ai-server:~# service gebo-ai status
● gebo-ai.service - Gebo.ai Service
   Loaded: loaded (/etc/systemd/system/gebo-ai.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-12-05 09:44:09 UTC; 1min 18s ago
     Main PID: 6192 (gebo-ai)
        Tasks: 53 (limit: 9376)
      Memory: 1.9G (peak: 1.9G)
         CPU: 20.685s
    CGroup: /system.slice/gebo-ai.service
            └─6192 /home/gebo-ai/gebo-ai/bin/gebo-ai

Dec 05 09:44:09 gebo-ai-server systemd[1]: Started gebo-ai.service - Gebo.ai Service.
root@gebo-ai-server:~#
```

Enabling the firewall to let you reach the gebo.ai web port:

```
#sudo ufw allow 12999/tcp
```

Ensure your server time is ok.

For security reasons, some clients checked the server has correct date settings, be sure to run the system with correct date/time and timezone.

Ensure all services are booted at system startup

```
#sudo systemctl enable qdrant
```

```
#sudo systemctl enable mongod
```

```
#sudo systemctl enable neo4j
```

```
#sudo systemctl enable gebo-ai
```

Gebo.ai on premise linux standard paths and infos:

The software runs as user **gebo-ai** user.

Main configuration file → **/etc/gebo-ai/application.properties**

Its home directory is **/home/gebo-ai** with software startup scripts and binaries.

Logs are placed in **/var/log/gebo-ai**.

The main log is:

/var/log/gebo-ai/ai.gebo.monolithic.app.log, periodically compressed and archived by the system.

Java garbage collection log → **/var/log/gebo-ai/gc.log***

Redirected startup console logs: standard output → **/var/log/gebo-ai/service.log** standard error → **/var/log/gebo-ai/service.err**

Upgrade Ubuntu gebo.ai installation

Check new releases here: <https://github.com/geboai/Gebo.ai/releases>

On the command shell do:

```
#sudo apt remove gebo-ai
```

```
#sudo apt install gebo-ai_<version>_amd64.deb
```

Check that the gebo-ai service is running with:

```
#sudo service gebo-ai status
```

Launch the service if is not running with:

```
#sudo service gebo-ai start
```

Data paths (to be backedup properly)

Backup the software home directory → **/home/gebo-ai**

Backup the software data directory → **/var/gebo-ai/data**

After software installation step

Go with your browser to: `http:<your server address>:12999/` create your administrator account and start using/administering the software for all the company.

Installing Gebo.ai on a windows server

Download actual gebo-ai version from the <https://gebo.ai/downloads/> or from <https://github.com/geboai/Gebo.ai/releases/> web page (select .msi format).

Hybrid docker compose/windows installation

To install an hybrid solution having mongodb/qdrant/neo4j installed by docker composer and the gebo.ai application running on a windows installation.

Use the following docker-compose.yml to launch the 4 server applications:

```
version: '3.1'
#=====
# This compose file creates a service for gebo.ai enterprise retrieve augmented generation
system
# and its required mongodb (database) and qdrant (vector database) services
# once runned simply open the address http://<your host address:12999/ with your
browser
# it will be required to create the administrative account and you'll be ready to setup and
# start immediately to share the application with your company or team.
services:
# =====
# MongoDB
# =====
# Bound only on loopback interface for security reasons
mongo:
  image: mongo
  restart: always
  ports:
    - "127.0.0.1:27017:27017"
  environment:
    MONGO_INITDB_ROOT_USERNAME: mongoroot
    MONGO_INITDB_ROOT_PASSWORD: mongopwd
  command: ["mongod", "--wiredTigerCacheSizeGB=1"]
```

```
# =====
# Qdrant
# =====
# Bound only on loopback interface for security reasons
qdrant:
  image: qdrant/qdrant
  restart: always
  ports:
    - "127.0.0.1:6333:6333"
    - "127.0.0.1:6334:6334"
  environment:
    QDRANT__SERVICE__API_KEY: ce7c85bc-f037-4c64-91d8-70335638329d
    QDRANT__STORAGE__OPTIMIZE_INDEXING: "true"
    QDRANT__STORAGE__RAM_DISK_SIZE: "256MB"
    QDRANT__STORAGE__MAX_SEGMENT_SIZE: "1000000"

neo4j:
  image: neo4j:5
  restart: always
  ports:
    - "127.0.0.1:7474:7474"
    - "127.0.0.1:7687:7687"
  environment:
    NEO4J_AUTH: "neo4j/neo4jmaster"
    NEO4J_server_memory_heap_initial_size: 1G
    NEO4J_server_memory_heap_max_size: 2G
    NEO4J_server_memory_pagecache_size: 512M
    NEO4JLABS_PLUGINS: '["apoc"]'
    NEO4J_apoc_export_file_enabled: "true"
    NEO4J_apoc_import_file_enabled: "true"
    NEO4J_apoc_import_file_use__neo4j__config: "true"
opensearch:
  image: opensearchproject/opensearch:latest
  container_name: opensearch
  environment:
    - discovery.type=single-node
    - OPENSEARCH_INITIAL_ADMIN_PASSWORD=dothesearch1973-Advanced
    - plugins.security.ssl.http.enabled=true
    - plugins.security.ssl.transport.enabled=true
  # consigliato in docker (evita bootstrap checks su memlock)
```

```
- bootstrap.memory_lock=true
# heap: regola in base alla RAM della macchina
- OPENSEARCH_JAVA_OPTS=-Xms1g -Xmx1g
ulimits:
  memlock:
    soft: -1
    hard: -1
  nofile:
    soft: 65536
    hard: 65536
ports:
  - "9200:9200"
  - "9600:9600"
```

Launch the docker-compose in the folder you've created the docker-compose.yml file running:

```
#docker-compose up -d
```

After these services are up and running you install Gebo.ai using the gebo-ai msi installer,

Go to the c:\Program Files\GeboAI\app\instance\config, copy the example-application.properties to application.properties (in the same folder), edit the file to provide connectivity information(s) for all the 3 different server applications (mongo, qdrant, neo4j) and restart the gebo-ai service from the windows services application.

After software installation step

Go with your browser to: [http:<your server address>:12999/](http://<your server address>:12999/) create your administrator account and start using/administering the software for all the company.

Installing MongoDB/Qdrant/Neo4J/Gebo.ai on the windows operating system.

This is actually the hardest installation to do, unfortunately most of the latest AI oriented software stack is not thought to run on windows but mostly on linux infrastructure or in docker infrastructure.

1st install actual version of MongoDB community edition →

<https://www.mongodb.com/try/download/community> (select community version and windows operating system). Create a mongo login profile in mongosh for the gebo.ai application (see how in the linux ubuntu section).

2nd install QDRANT Vector database → <https://github.com/qdrant/qdrant/releases/> (select the pc-windows-msvc*.zip version) . Create a qdrant api-key for the gebo.ai application. You can set a qdrant configuration coherent with the one in the linux ubuntu section. Unfortunately there are no QDRANT distributions running as a service, so it will run as a console application.

3rd install Neo4J graph database → <https://neo4j.com/deployment-center/?ref=subscription#community> (select the community edition), install the application and create/test an account protected by password in the neo4j system.

After these services are up and running you install Gebo.ai using the gebo-ai msi installer,

Go to the c:\Program Files\GeboAI\app\instance\config, copy the example-application.properties to application.properties (in the same folder), edit the file to provide connectivity information(s) for all the 3 different server applications (mongo, qdrant, neo4j) and restart the gebo-ai service from the windows services application.

After software installation step

Go with your browser to: [http:<your server address>:12999/](http://<your server address>:12999/) create your administrator account and start using/administering the software for all the company.

“Monolithic version” base installation components & integration with 3rd party systems.

The gebo.ai monolithic version software is an application that runs on linux/unix/windows and on containerized systems using java virtual machines 17 or higher (preferable the 21 or 22 versions) runtimes.

Minimum software stack for the installation

- Jvm 21 or 22 (already shipped in bundle with the geboai/gebo.ai docker container).
- A reachable **MongoDB** installation (also a common edition is OK)
- A reachable Vector database, preferably **QDRANT** that the more tested one.
- A reachable **Neo4J** graph database (optional but suggested) for Graphrag and Knowledge extraction

MongoDB is the only part of the infrastructure that must be configured dealing directly with the software configuration files, almost all the other integrations or required infrastructure parts can be configured from the administrative UI.

Connectivity with cloud generative AI providers or local generative AI server

The software can use the following cloud or on-premise generative AI providers/servers, you require at least to configure 2 default models for the software:

- a “chat large language model” (like GPT4o/O1/Claude 3.X/Llama 3.X or others) → this is the chatbot that will perform generative AI session elaboration and output.
- an “embedding model” like OpenAi text-embedding-3-large or NVIDIA/NV-Embed-v2 or others. This model is responsible for transforming document contents on their semantic numerical vector representation to support data retrieval.

Supported providers (and some supported models examples)

From networking/technical point of view those of the following cloud providers chosen must be configured to have full accessibility from the server where Gebo.ai is installed on. In other words if you chose to use OpenAI or Xai or other suppliers, at least https networking with those domains have to be permitted on the company firewalls.

All the cloud or on premise generative AI providers or servers can be directly configured using the software administrative UI, inserting directly in the user interface api key or other company subscription credentials.

The credentials management uses cryptographic algorithms to let those access informations being securely stored and unreadable even if the database is hacked (go to the security considerations sections).

- OpenAi, chat models (suggested, best multilanguage)

- Anthropic, chat models: Claude
- MistralAI, chat models: Mistral Large/Codestral/Mistral mini embedding models: Mistral Embed (English only).
- Xai, chat models: Grok, embedding models: embedding-beta
- Groq provider, chat models: Meta Llama 3.1 3.2 3.3 8b/70b
- Regolo.ai provider (Italian player developing interesting offers)
- Nvidia Nim/Cloud platform (mostly for models trials purposes), chat models: <https://build.nvidia.com/explore/reasoning> embedding models: <https://build.nvidia.com/explore/retrieval>
- Supported soon: Google AI infrastructures.
- Other infrastructures compatible with OpenAI API can be tested with the generical OpenAI API connection profile.

Supported local/lan generative AI server

- Ollama: mostly suggested for models trials for its easy setup, does not scale well to an enterprise use.
- vLLM: the “go to” solution for on premise artificial intelligence, scales to multiple devices/servers taking the most from actual Nvidia/Amd hardware.
- All other on premise applications providing full OpenAI compatible REST api.

Connectivity with company used on premise or cloud systems

1st connectivity with various “content systems” or “file sharing systems”

The software has an administrative area where users with administrative rights can configure integrations with all the Cloud or “On premise” content management systems or control version systems or files/documents sharing systems.

2nd composing knowledge bases

Administrative users will compose huge knowledge bases adding documents chosen from all the available CMS/shared filesystems/versioning systems to be available via retrieve augmented generation while Gebo.ai users interact with chat bots.

The server where Gebo.ai is installed must have access to all of those network resources.

All the content management services/version management services/sharing filesystem services/database services providers or servers can be directly configured using the software administrative UI, inserting directly in the user interface all the required credentials.

The credentials management uses cryptographic algorithms to let those access informations being securely stored and unreadable even if the database is hacked (go to the security considerations sections).

Gebo.ai detailed configuration

The application(s) distributed in the docker environment with an application.yml configuration that already works in the context where the provided docker-compose.yml is used.

The internal path on the container is /opt/gebo.ai/config/application.yml, if necessary it can be extracted from the standard container running:

```
docker cp <container name>:/opt/gebo.ai/config/application.yml ./application.yml
```

or it can be edited directly inside the container.

For on premise “non docker” installations there is a shipped application.properties file under /etc/gebo-ai/ in linux or in c:\Program Files/GeboAI/app/instance/config in windows operating system.

All the following chapters mention configuration keys and values that already exist in this file or that can be added to change the software behavior.

Be careful when handling the yml file format to use a software with proper syntax management like visual code with the yml format plugin. This format requires proper formatting to be accepted by the application.

Use an application.properties format if not familiar with the yaml format, you can simply use this translator: <https://mageddo.com/tools/yaml-converter> to convert the application.yml to an application.properties file and use this in place of the first once removed.

GEBO_HOME environment variable or system property

When the application is started it requires a GEBO_HOME environment variable (already set in the docker container to /opt/gebo.ai/home or by installers in “on premise” installations) .

This variable must point to an existing folder that is usable to store minimal application internal configurations. It is convenient to backup this folder periodically.

To ensure coherency on persistence using the docker image we suggest to mount this folder to an existing folder on the host system and backup it periodically.

GEBO_WORK_DIRECTORY environment variable or system property or interactive admin UI setting.

The application requires the setting of a “work directory” used to store its working files, It is convenient to backup this folder periodically.

The work directory can be handle in 2 different way:

- Work directory interactively set at gebo.ai interactive setup by the administrative user using the administrator user interface, behavior configured when the configuration key:

ai.gebo.config.setupConfiguresWorkdir is set to true. In this state the work directory path is configured in the MongoDB system.

- Work directory configured via configuration file, behavior set with:
ai.gebo.config.setupConfiguresWorkdir is set to false with
GEBO_WORK_DIRECTORY environment or system property pointing to the chosen work directory (that is required to be an existing and empty folder).

In the containerized docker distribution the system work with this 2nd type of configuration with work directory being set to /opt/gebo.ai/work folder.

To ensure coherency on persistence using the docker image we suggest to mount this folder to an existing folder on the host system and backup it periodically.

MongoDB connectivity settings

This base configuration is mandatory and required for the application to start, the configuration is in the following application.yml configuration keys (or if you choose to use application.properties format simply change “:” with “=” in the following keys configuration)

ai.gebo.mongodb.enabled: true

ai.gebo.mongodb.databaseName: gebo-ai

ai.gebo.mongodb.connectionString: mongodb://<user>:<pwd>@localhost:27017/gebo-ai?authSource=admin

The connection string grammar reference is standard for MongoDB configurations and is documented here: <https://www.mongodb.com/docs/manual/reference/connection-string/>

If you are using the docker-compose launch system the provided docker-compose.yml file sets up MongoDB already coherent with these settings shipped with the provided configuration.

MongoDB is used as NOSQL database for configurations, application data and much more, so it must be considered part of the “persistent storage class” of the application, managed taking into account security and under backup management.

QDRANT or other vector database configuration

This base configuration can be provided from the application.yml configuration file (or if you choose to use application.properties format simply change “:” with “=” in the following keys configuration), however if is not provided a clean new installation started up without it and let the administrator user configure it interactively in the administration UI “setup” section.

Configuration keys:

ai.gebo.vectorstore.use: QDRANT

ai.gebo.vectorstore.qdrant.host: 127.0.0.1

ai.gebo.vectorstore.qdrant.port: 6334

ai.gebo.vectorstore.qdrant.tls: false

ai.gebo.vectorstore.qdrant.apiKey: <api key value>

The vector database is a mandatory component of the system when we have configured at least one large language model with “embedding” capabilities, Gebo.ai create separate vector databases connection for each “embedding model” and configured Knowledge base, so once configured and your company started to use the retrieve augmented generation system it has to be considered as a part of the “persistent storage class” in the application stack keeping it secure and with managed backup.

Neo4J database configuration

The graph database configuration can be provided from the application.yml configuration file (or if you choose to use application.properties format simply change “:” with “=” in the following keys configuration).

Configuration keys:

ai.gebo.neo4j.enabled: <true to enable, false to disable>

spring.neo4j.uri: bolt://localhost:7687

spring.neo4j.authentication.username: <neo4j user name>

spring.neo4j.authentication.password: secret

OpenSearch database configuration

The opensearch database configuration can be provided from the application.yml configuration file (or if you choose to use application.properties format simply change “:” with “=” in the following keys configuration).

Configuration keys:

ai.gebo.opensearch.enabled: true

ai.gebo.opensearch.protocol: https

ai.gebo.opensearch.host: 127.0.0.1

ai.gebo.opensearch.port: 9200

ai.gebo.opensearch.username: admin

ai.gebo.opensearch.password: <your opensearch admin password>

Company shared filesystem

Company shared filesystems paths, due to their delicate handling, must first being configured to be reachable from the Gebo.ai software.

For the docker compose or docker easyinstall image:

If you are using the provided docker-compose.yml file to install/launch the system you need to mount your filesystems under the **/opt/gebo.ai/shares** on the linux docker installation using the standard documer (or integrating it in docker-compose.yml) “volume mounting” commands.

Gebo.ai — User & Admin screens Software Guide

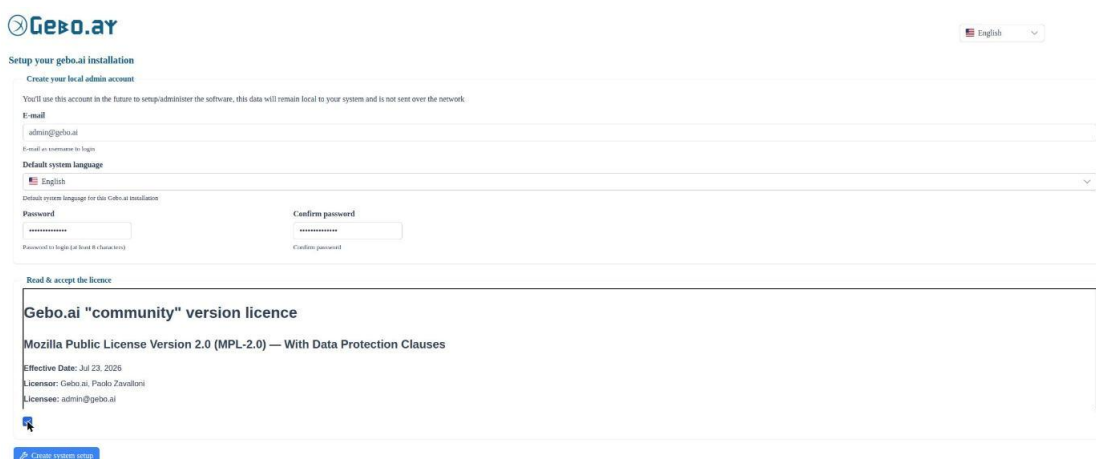
A visual, screen-by-screen walkthrough of Gebo.ai's setup wizard, admin console, and chat interface, captured from a fresh local installation — from creating the first administrator account through to a working, LLM-backed chat conversation.

First-run setup

The very first time you open Gebo.ai — at `http://<host>:12999/` — you land on a one-time setup screen. Nothing else in the application is reachable until this is completed.

Create your local admin account

Email (used as the login username), a default system language, a password, and acceptance of the license. This account is stored locally — the setup screen explicitly notes the data never leaves your system.

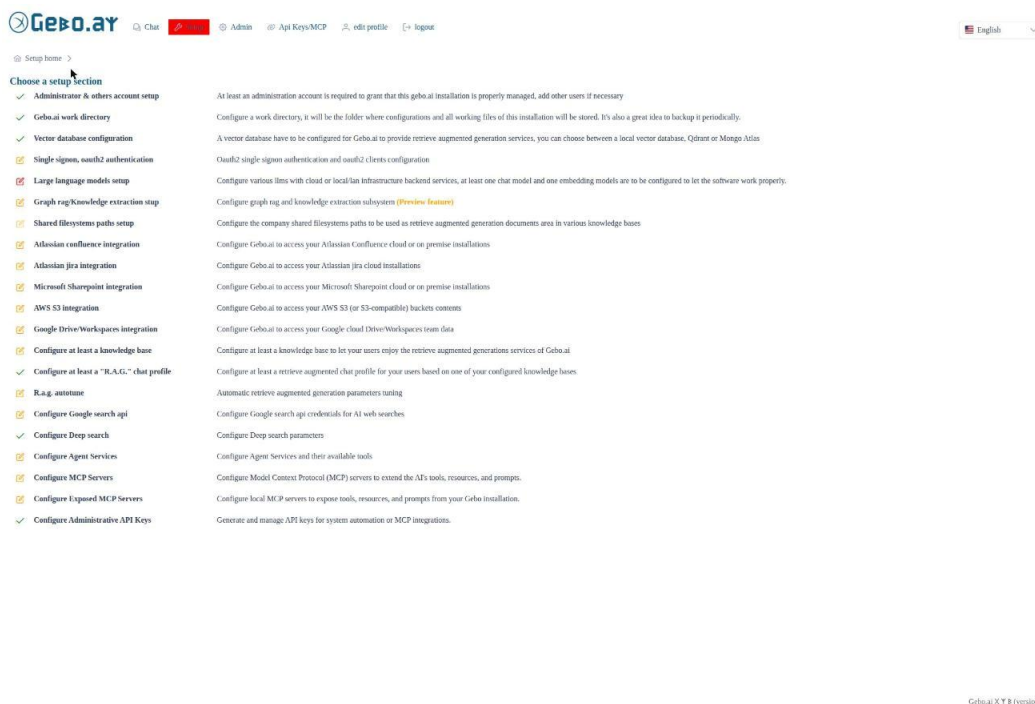


The screenshot displays the Gebo.ai setup interface. At the top left is the Gebo.ai logo, and at the top right is a language dropdown menu set to 'English'. The main heading is 'Setup your gebo.ai installation'. Below this, there are two sections: 'Create your local admin account' and 'Read & accept the license'. The 'Create your local admin account' section includes a note that the account is for local use only, followed by input fields for 'E-mail' (pre-filled with 'admin@gebo.ai'), 'Default system language' (set to 'English'), 'Password', and 'Confirm password'. The 'Read & accept the license' section shows the 'Gebo.ai "community" version licence' and the 'Mozilla Public License Version 2.0 (MPL-2.0) — With Data Protection Clauses'. It also lists the effective date as 'Jul 23, 2026', the licensor as 'Gebo.ai, Paolo Zavalloni', and the licensee as 'admin@gebo.ai'. A 'Create system user' button is located at the bottom left of the license section.

Submitting this form redirects to the LLM setup wizard with a fully authenticated top navigation bar (Chat / Setup / Admin / Api Keys/MCP / profile / logout) — that's the confirmation the account now exists.

The setup checklist — your map of the whole application

Exiting the LLM wizard drops you on the setup checklist: 21 configuration sections, each with a status icon (green check = done/optional-with-default, pencil = still to configure). This single page is effectively the site map for every administrative capability Gebo.ai has. We revisit it in full in [Part 4](#).



Connecting your first LLM provider

Nothing else in Gebo.ai works — not chat, not knowledge bases, not agents — until at least one chat model and one embedding model are configured. The "easy setup" wizard gets you there in under a minute for any of its built-in providers.

Easy initial setup — pick a vendor, paste a key

Six one-click vendor shortcuts (Regolo.ai, OpenAI, Xai, NVidia, Mistral.ai, OpenRouter.ai) sit alongside links to the provider's website and its API-key purchase page. Choosing a vendor reveals a credentials form: a description, a username, and the API key itself.

The screenshot shows the 'Setup Large language models' page in the Gebo.ai interface. The 'Easy initial setup/configuration fix' tab is active. It features a section for choosing a vendor (Regolo.ai, OpenAI, Xai, NVidia, Mistral.ai, OpenRouter.ai) and a 'Provider web site' link. Below this, there's a form for creating or using existing API key credentials. The form includes fields for 'Credentials description', 'User name', and 'New api key', with a 'Save' button. A 'Suggested settings' section is also visible at the bottom.

Gebo.ai

Chat Setup Admin API Keys MCP edit profile login

English

← Exit setup checklist

Setup Large language models

Easy initial setup/configuration fix Extended infrastructure/suppliers choices

*Choose your large language model vendor

Regolo.ai OpenAI Xai NVidia Mistral.ai OpenRouter.ai

Provider web site

<https://regolo.ai>

Purchase provider's credit

<https://dashboard.regolo.ai/api-keys>

Use or create api key credentials

*Use existing or new

Existing credentials New credentials

*Credentials description

regolo.ai credentials

*User name

*New api key

Save

Suggested settings

Create and activate

Gebo.ai X.T.B (version: 1.0.2.1-SNAPSHOT)

API keys are entered directly into this form and are stored encrypted server-side — never handled by anything outside your own browser session.

Extended infrastructure/suppliers choices

The second tab of the same wizard lists every supported backend individually, each collapsible: Regolo.ai, OpenAI, Anthropic, Deepseek, Xai, NVidia, Mistral.ai, Groq, OpenRouter.ai, Ollama, vLLM on-premise infrastructure, and any generic OpenAI-compatible API provider. This is the path for local/self-hosted models (Ollama, vLLM) or providers without a one-click shortcut.

The screenshot shows the 'Setup Large language models' page in the Gebo.ai interface, with the 'Extended infrastructure/suppliers choices' tab selected. It displays a list of supported providers, each with a dropdown arrow for further configuration. The providers listed are Regolo.ai, OpenAI, Anthropic, Deepseek, Xai, NVidia, Mistral.ai, Groq, OpenRouter.ai, Ollama, vLLM on premise infrastructure, and OpenAI compliant API provider.

Gebo.ai

Chat Setup Admin API Keys MCP edit profile login

English

← Exit setup checklist

Setup Large language models

Easy initial setup/configuration fix Extended infrastructure/suppliers choices

Regolo.ai

OpenAI

Anthropic

Deepseek

Xai

NVidia

Mistral.ai

Groq

OpenRouter.ai

Ollama

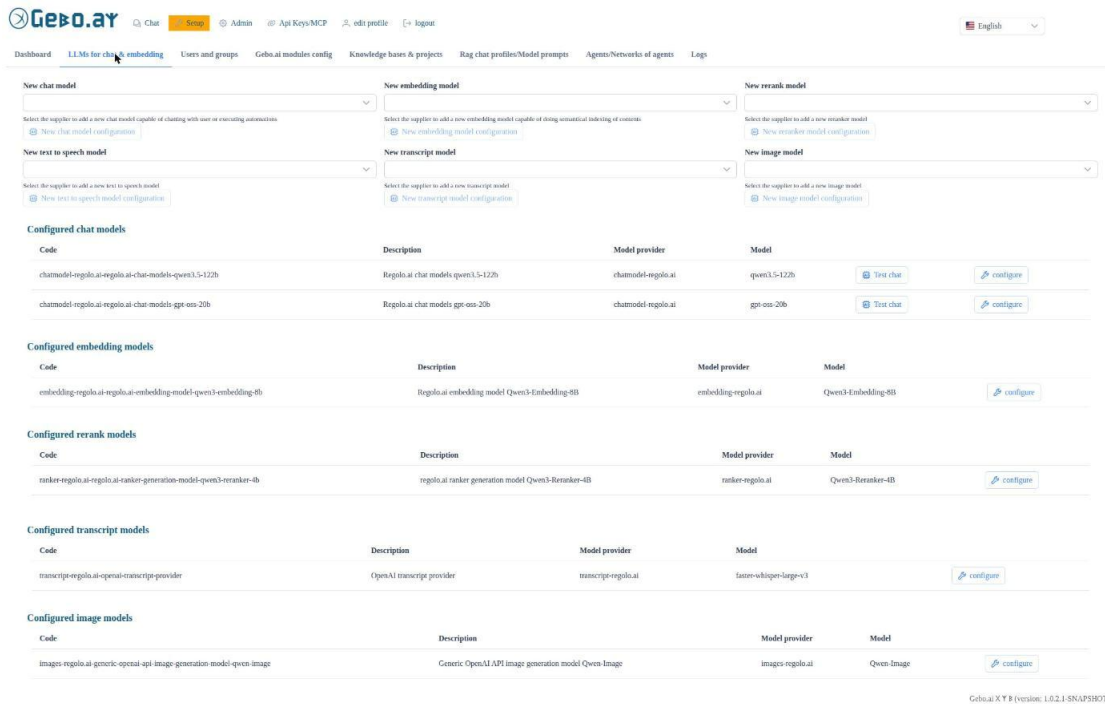
vLLM on premise infrastructure

OpenAI compliant API provider

Gebo.ai X.T.B (version: 1.0.2.1-SNAPSHOT)

Result: six model types provisioned from one API key

After activating Regolo.ai, **Admin → LLMs for chat & embedding** shows what the wizard actually built: two chat models (qwen3.5-122b, gpt-oss-20b), an embedding model (Qwen3-Embedding-8B), a reranker (Qwen3-Reranker-4B), a transcription model (faster-whisper-large-v3), and an image-generation model (Qwen-Image) — all from a single provider credential, using Regolo.ai's suggested defaults.



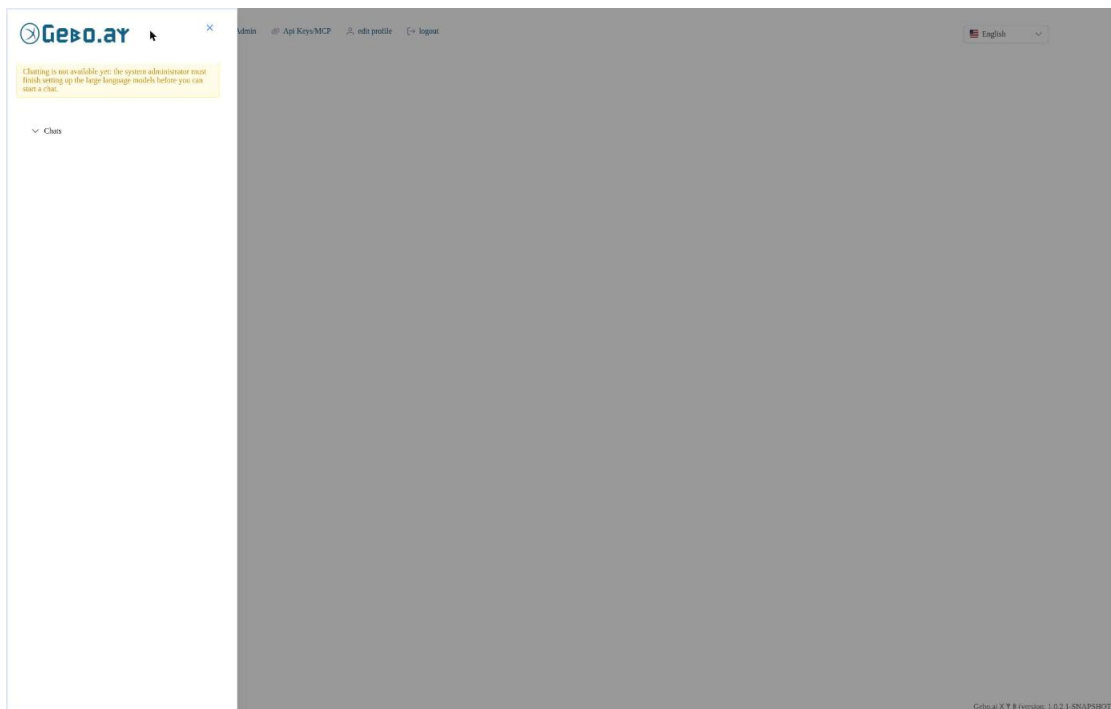
Every one of these model types maps directly to a Gebo.ai feature: chat models power conversations, embedding models power RAG retrieval, the reranker improves retrieval relevance, the transcript model powers speech-to-text, and the image model powers in-chat image generation.

Chat

The Chat screen behaves differently depending on whether an LLM is configured — a good illustration of how tightly Gebo.ai gates functionality on setup completeness.

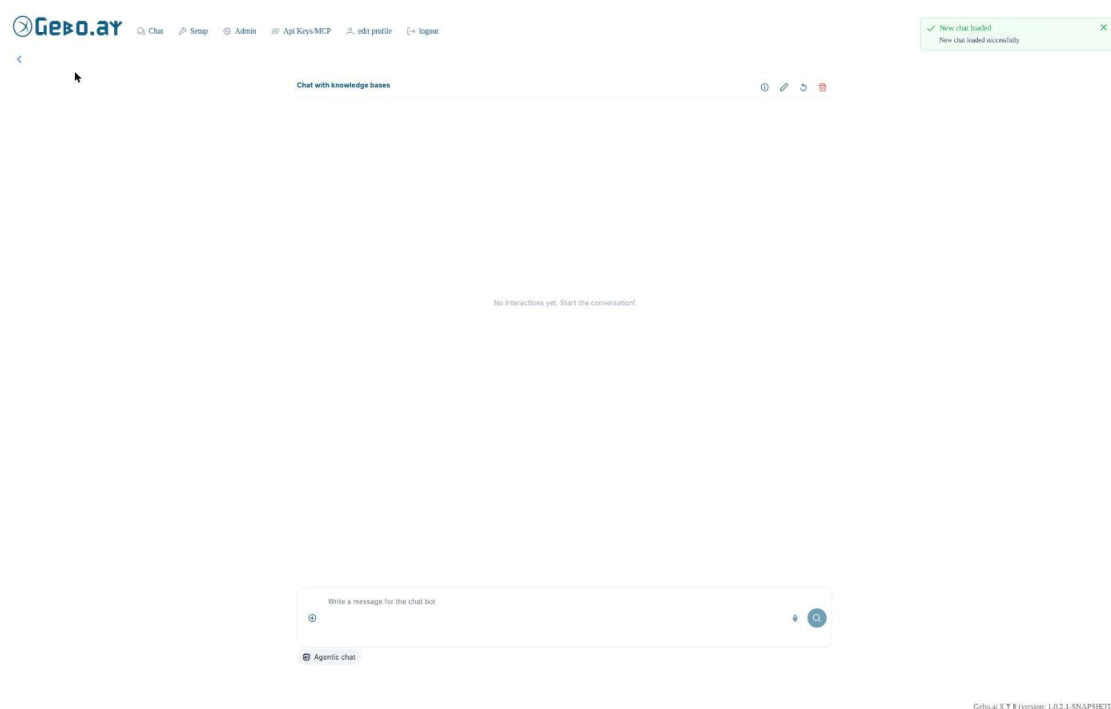
Before: no LLM configured

A clear, actionable message instead of a broken chat box.



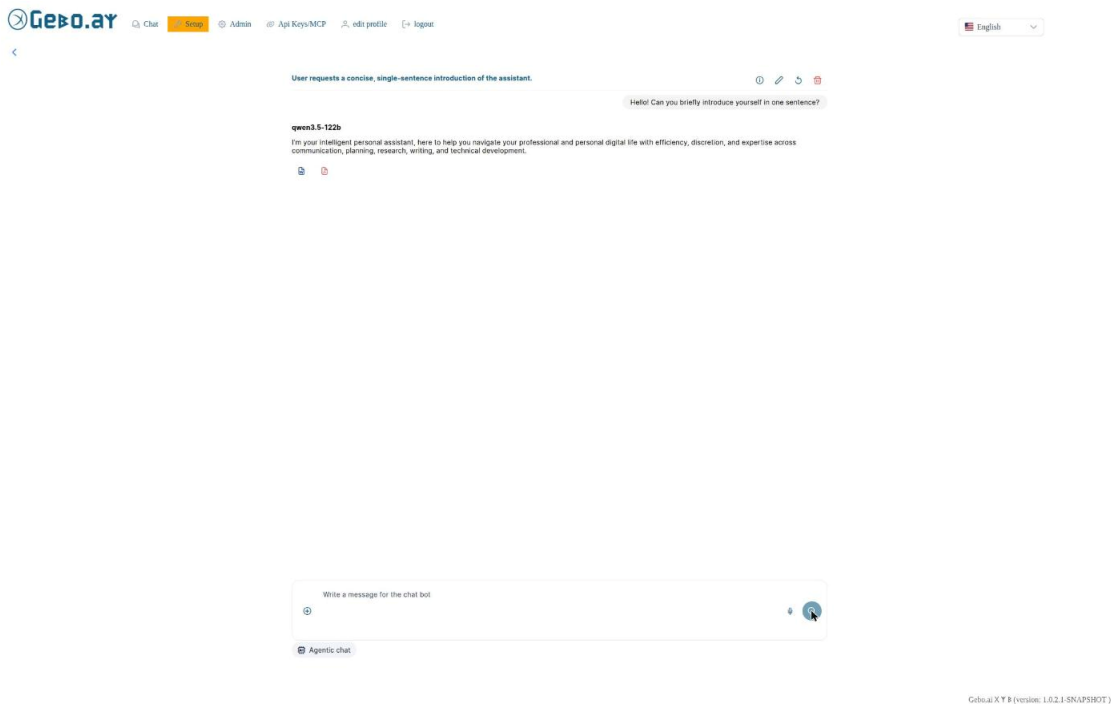
After: chat is live

"Chat with knowledge bases" — the default RAG chat profile — opens a normal chat window, with an "Agentic chat" toggle and a mic button for voice input.



A real conversation

A round-trip through the configured qwen3.5-122b model. Note the AI-generated conversation title ("User requests a concise, single-sentence introduction of the assistant") that replaced the generic chat name after the first exchange — Gebo.ai titles conversations automatically.

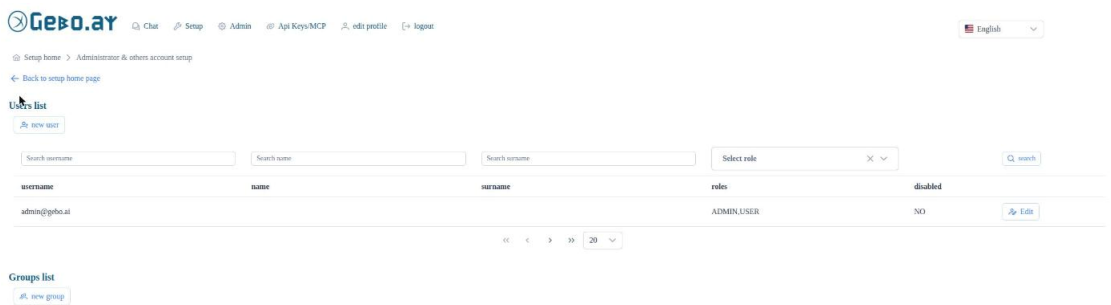


The full setup checklist

Every one of the 21 sections from the checklist in Part 1, screen by screen. Several sections are gated behind prerequisites — the wizard blocks navigation into them rather than showing a broken screen, which is worth knowing about as an admin.

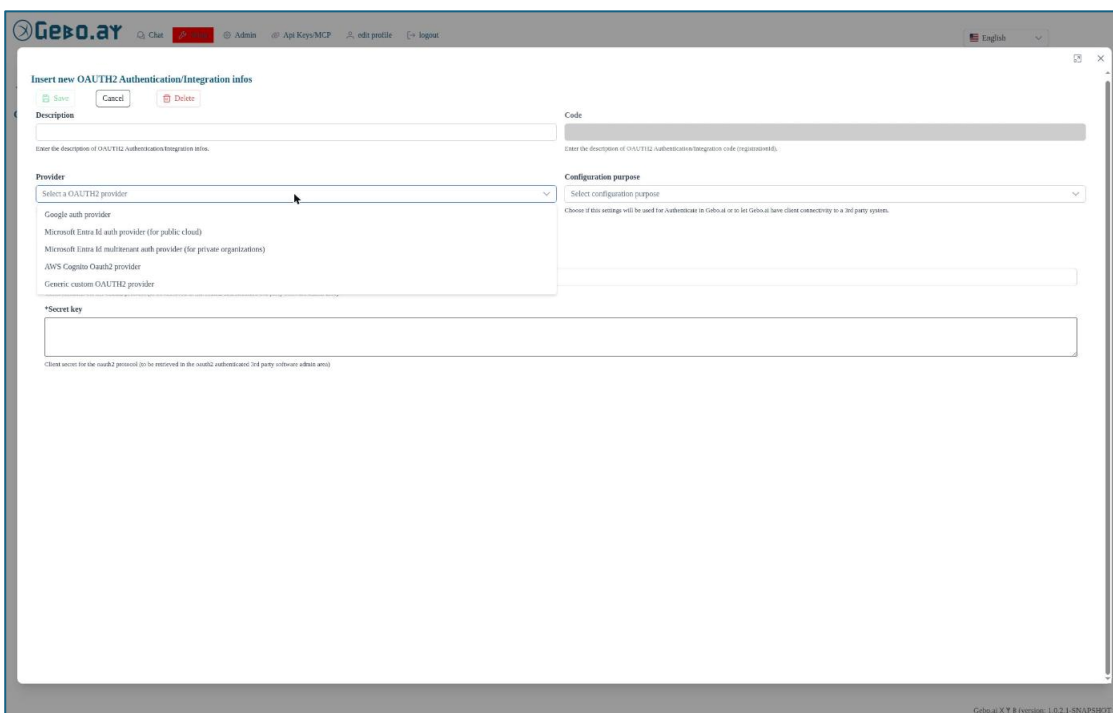
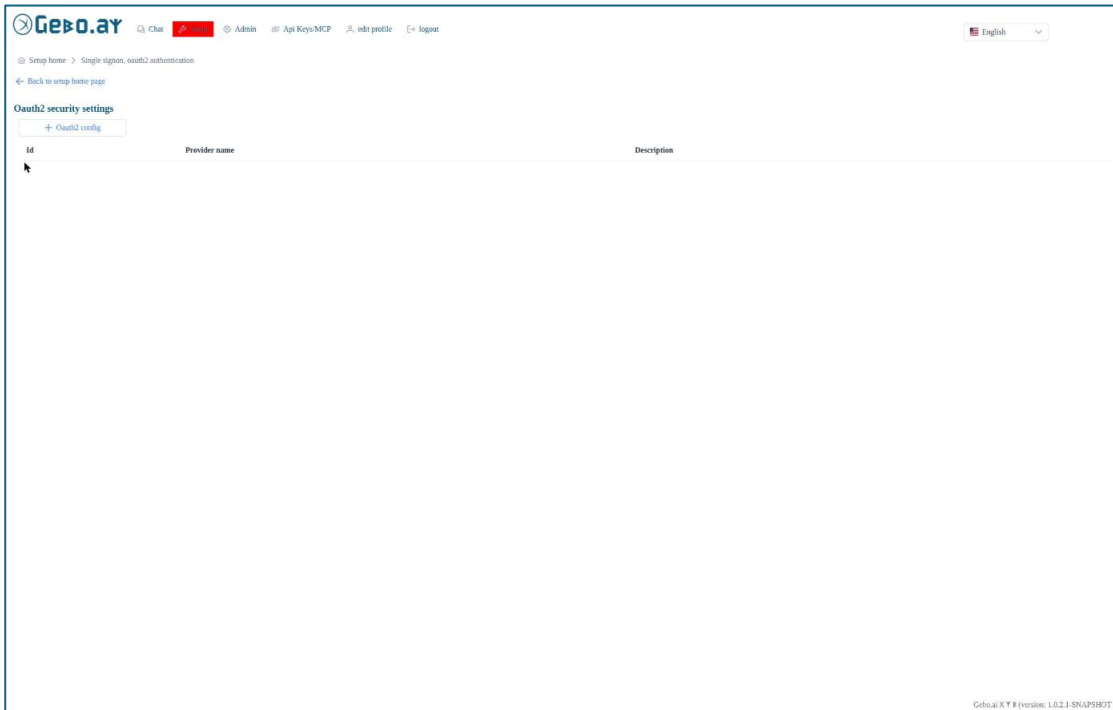
Administrator & others account setup

The same users/groups management surface also reachable from Admin → Users and groups: search by username/name/surname/role, edit roles, disable accounts, and manage groups.



Single sign-on, OAuth2 authentication

An empty registrations list plus the "Insert new OAUTH2 Authentication/Integration infos" form. The provider dropdown reveals the five built-in options: Google, Microsoft Entra ID (public cloud), Microsoft Entra ID multitenant (private orgs), AWS Cognito, and a generic OAuth2 provider — which is how KeyCloak or any other OIDC-compliant IdP gets connected.



The form flags a security warning: OAuth2 client secrets are encrypted at rest, but a misconfiguration here can lock users out — handle with care.

Every provider shares four top-level fields — Description, Code (auto-generated registration id), Provider, and Configuration purpose — plus a "OAuth2 registered client" block with the standard *Client ID and *Secret key. What changes per provider is an optional "OAuth2 provider custom attributes" block underneath, shown below for each of the five.

Configuration purpose — the same for every provider

Independent of which provider is selected: **Authentication** registers the provider as a sign-in option on Gebo.ai's own login screen, while **Integration client** registers it purely so Gebo.ai can act as an OAuth2 client against a 3rd-party system (e.g. to call an API on the user's behalf) without it appearing as a login method.

The screenshot shows the 'Insert new OAuth2 Authentication/Integration info' form in the Gebo.ai admin interface. The form is divided into two main sections: 'OAuth2 provider URLs' and 'OAuth2 registered client'.

OAuth2 provider URLs section:

- Description:** A text input field with a placeholder 'Enter the description of OAuth2 authentication/integration info.' and a 'Code' field with a placeholder 'Enter the description of OAuth2 authentication/integration code (registrationid)'.
- Provider:** A dropdown menu with 'Generic custom OAuth2 provider' selected.
- Configuration purpose:** A dropdown menu with 'Authentication' selected.
- OAuth2 provider URLs:** A section with four input fields: 'Issuer URI', 'Token URL', 'Introspection URI', and 'User info URL'. Each field has a placeholder 'Enter the OAuth2 [field name] URL of the provider'.

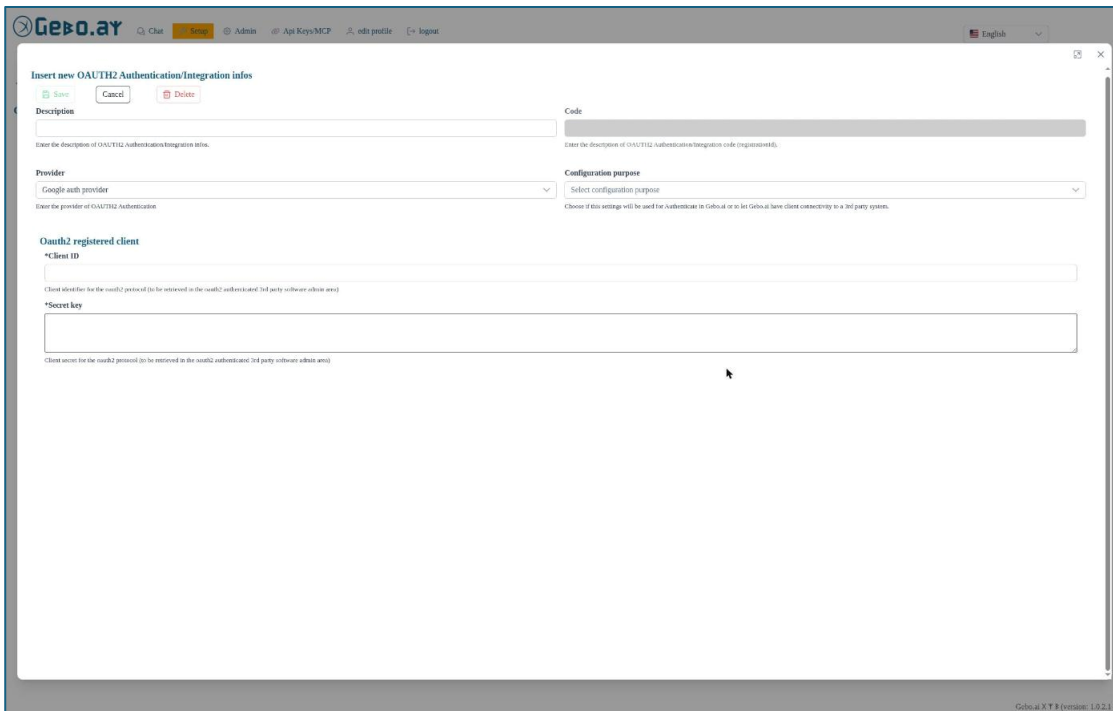
OAuth2 registered client section:

- *Client ID:** A text input field with a placeholder 'Client identifier for the oauth2 protocol (to be entered in the oauth2 authorized 3rd party software admin area)'.
- *Secret key:** A text input field with a placeholder 'Client secret for the oauth2 protocol (to be entered in the oauth2 authorized 3rd party software admin area)'.

The form includes 'Save', 'Cancel', and 'Delete' buttons at the top left. The Gebo.ai logo and navigation menu are visible at the top of the browser window.

Google auth provider

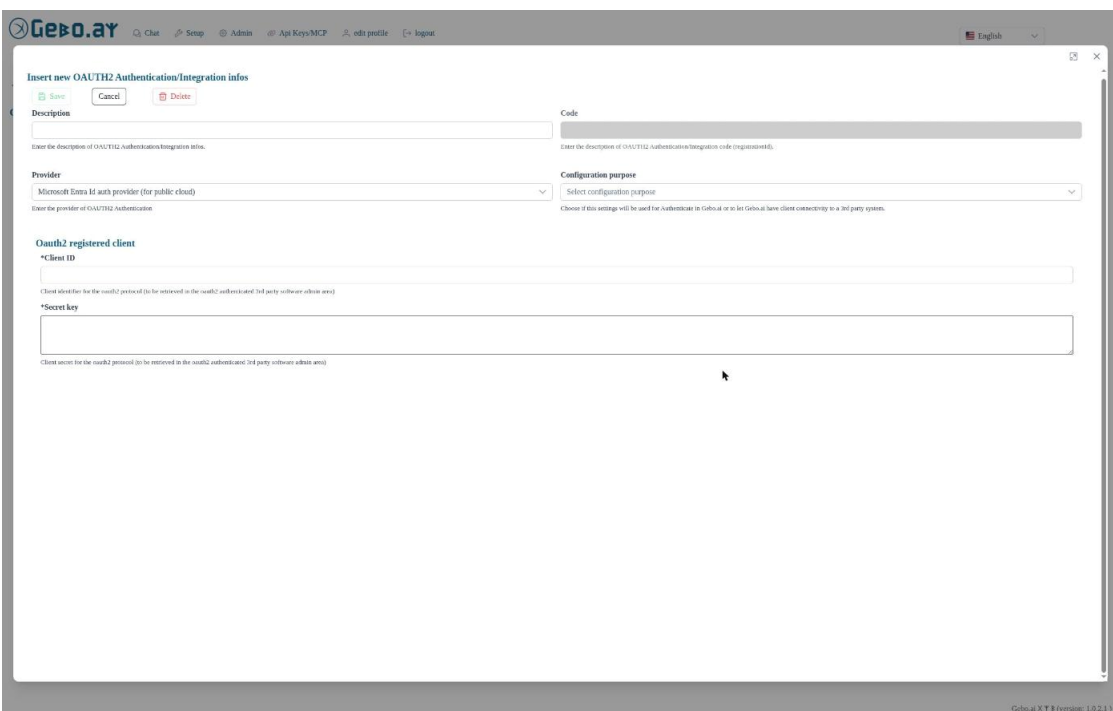
No custom attributes — just the standard Client ID/Secret pair from a Google Cloud OAuth 2.0 Client ID.



Gebo.ai X.T.F (version: 1.0.2.1)

Microsoft Entra ID — public cloud

Also no custom attributes beyond Client ID/Secret — this variant targets the standard multi-org common/public-cloud Entra endpoint, so there's nothing tenant-specific to fill in.



Gebo.ai X.T.F (version: 1.0.2.1)

Microsoft Entra ID multitenant (private organizations)

Adds one custom attribute: `tenantId` — the Microsoft Entra organization's tenant id, required so Gebo.ai authenticates against that specific org's directory instead of the public multi-tenant endpoint. The field renders with a red validation outline until filled in, since it's required for this provider.

Insert new OAUTH2 Authentication/Integration info

Save Cancel Delete

Description
Enter the description of OAUTH2 Authentication/Integration info.

Provider
Microsoft Entra ID multi-tenant auth provider (for private organizations)
Enter the provider of OAUTH2 Authentication.

Code
Enter the description of OAUTH2 Authentication/Integration code (optional).

Configuration purpose
Select configuration purpose
Choose if this settings will be used for Authenticate in Gebo.ai or to let Gebo.ai have client connectivity to a 3rd party system.

OAuth2 registered client
*Client ID
Client identifier for the OAuth2 protocol (to be entered in the OAuth2 authorized 3rd party software admin area)
*Secret key
Client secret for the OAuth2 protocol (to be entered in the OAuth2 authorized 3rd party software admin area)

OAuth2 provider custom attributes
tenantId
Microsoft Entra organization tenantId

Gebo.ai X.T.T (version: 1.0.2.1)

AWS Cognito

Adds three custom attributes: cognitoDomain (the AWS Cognito domain), userPoolId (the AWS Cognito user pool), and region (the AWS region the pool lives in) — everything Gebo.ai needs to resolve Cognito's hosted-UI and token endpoints without hardcoding them.

Insert new OAUTH2 Authentication/Integration info

Save Cancel Delete

Description
Enter the description of OAUTH2 Authentication/Integration info.

Provider
AWS Cognito OAuth2 provider
Enter the provider of OAUTH2 Authentication.

Code
Enter the description of OAUTH2 Authentication/Integration code (optional).

Configuration purpose
Select configuration purpose
Choose if this settings will be used for Authenticate in Gebo.ai or to let Gebo.ai have client connectivity to a 3rd party system.

OAuth2 registered client
*Client ID
Client identifier for the OAuth2 protocol (to be entered in the OAuth2 authorized 3rd party software admin area)
*Secret key
Client secret for the OAuth2 protocol (to be entered in the OAuth2 authorized 3rd party software admin area)

OAuth2 provider custom attributes
cognitoDomain
AWS Cognito domain
userPoolId
AWS Cognito user pool
region
AWS Region

Gebo.ai X.T.T (version: 1.0.2.1)

Generic custom OAUTH2 provider

The escape hatch for anything not built in — KeyCloak, Okta, Auth0, a self-hosted OIDC provider, etc. Instead of a handful of named attributes, it exposes the full set of OAuth2/OIDC endpoint URLs directly: **Issuer Uri, Authorization URL, Token URL, User info URL,**

Introspection Uri, and **Jwk Set Uri** — all optional individually, but between them enough to hand-wire any spec-compliant provider that doesn't have a dedicated preset above.

Gebo.ay

Chat

Settings

Admin

API Keys MCP

edit profile

login

English

Insert new OAUTH2 Authentication/Integration info

Save

Cancel

Delete

Description

Enter the description of OAUTH2 Authentication/Integration info.

Provider

Generic custom OAUTH2 provider

Enter the provider of OAUTH2 Authentication.

OAUTH2 provider URLs

Issuer URI

Enter the OAUTH2 Issuer URI of the provider

Token URI

Enter the OAUTH2 Token URI of the provider

Introspection Uri

Enter the OAUTH2 Introspection URI of the provider

Code

Enter the description of OAUTH2 Authentication/Integration code (optional).

Configuration purpose

Select configuration purpose

Choose if this settings will be used for Authentication in Gebo.ay or to let Gebo.ay have client connectivity to a 3rd party system.

Authorization URI

Enter the OAUTH2 Authorization URI of the provider

User info URI

Enter the OAUTH2 User info URI of the provider

Jwk Set Uri

Enter the OAUTH2 Jwk Set Uri of the provider

Oauth2 registered client

*Client ID

Client Identifier for the oauth2 protocol (to be entered in the oauth2 authorized 3rd party software admin area)

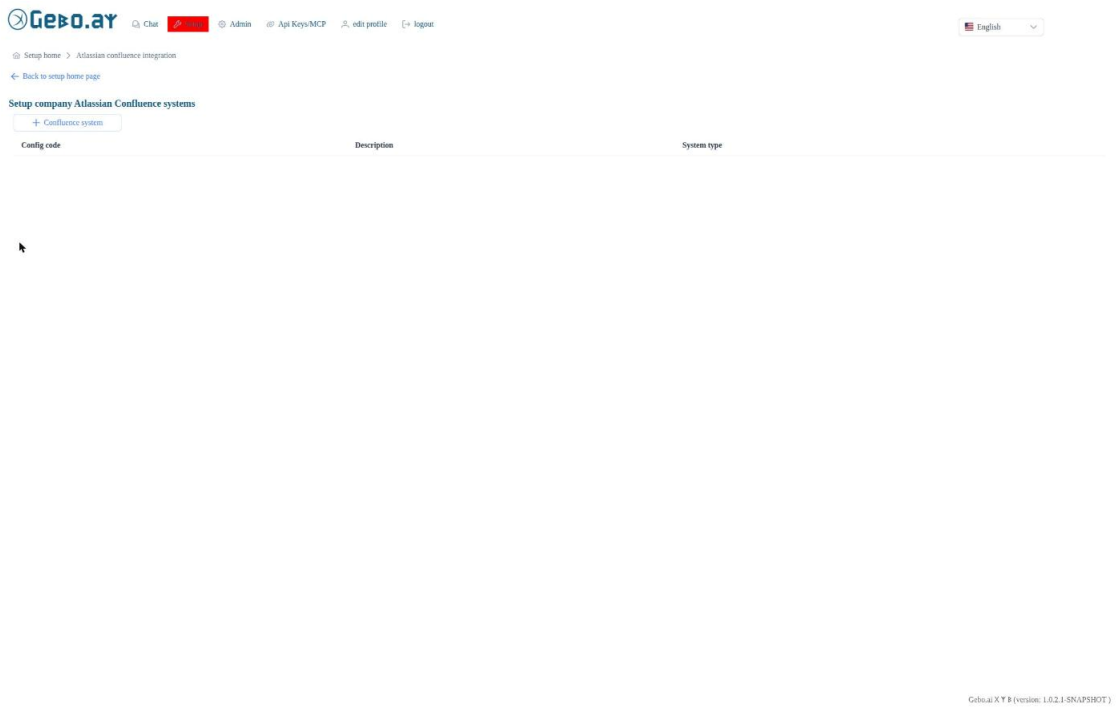
*Secret key

Client secret for the oauth2 protocol (to be entered in the oauth2 authorized 3rd party software admin area)

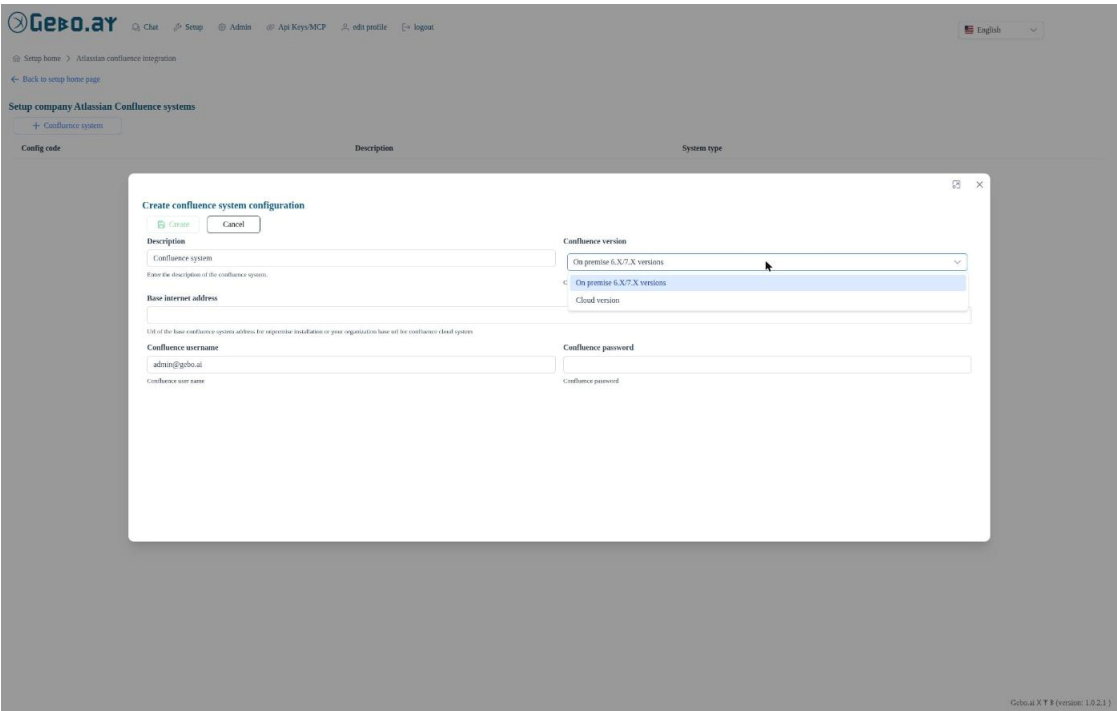
Gebo.ay KTF (version: 1.0.2.1)

Atlassian Confluence integration

An empty "Config code / Description / System type" list with a "Confluence system" button to register one or more Confluence cloud or on-premise sources.

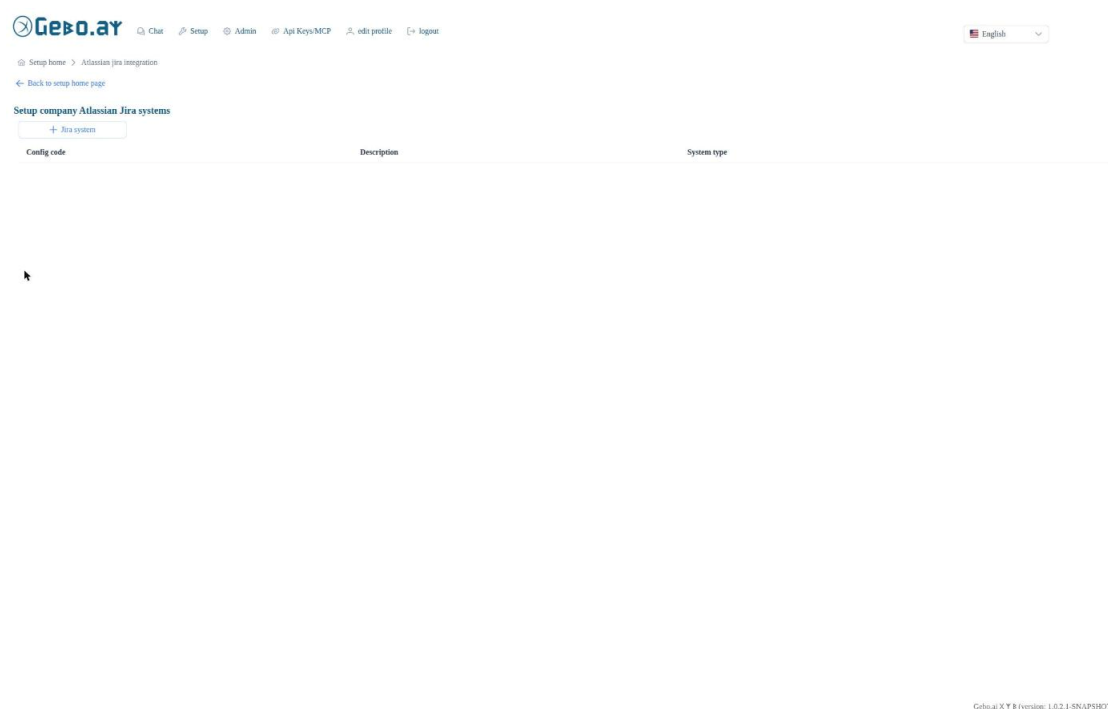


The "+ Confluence system" form: Description, a **Confluence version** toggle (On premise 6.X/7.X versions / Cloud version), Base internet address, Confluence username, and Confluence password — the same four fields regardless of which version is selected.

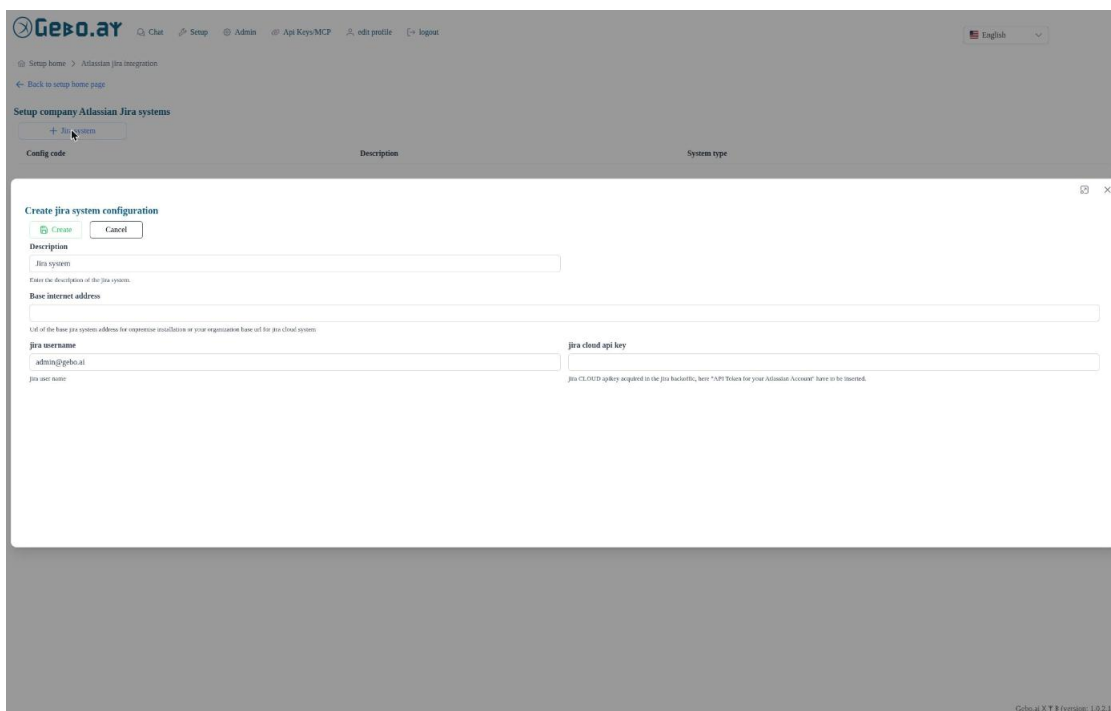


Atlassian Jira integration

Same pattern as Confluence, for Jira instances.

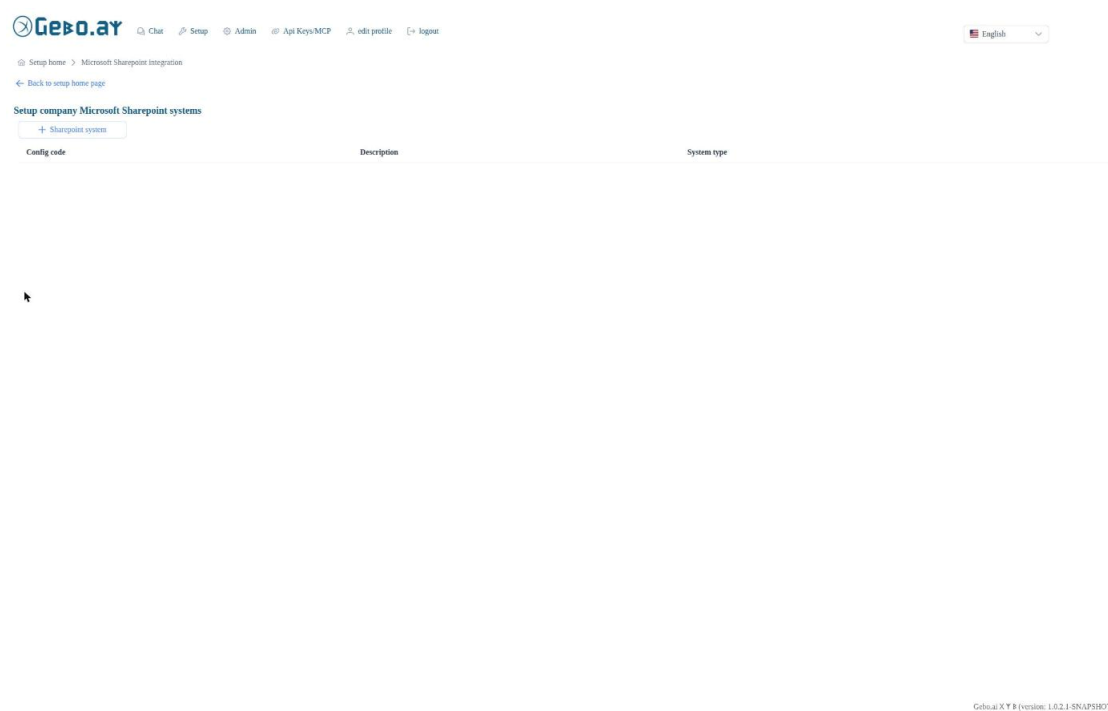


Jira's add form is simpler — no version toggle. Just Description, Base internet address, Jira username, and a **Jira cloud api key**: the help text is explicit that this must be the "API Token for your Atlassian Account" generated in the Jira backoffice, not a plain password.



Microsoft SharePoint integration

Registers SharePoint/OneDrive cloud or on-premise sources.

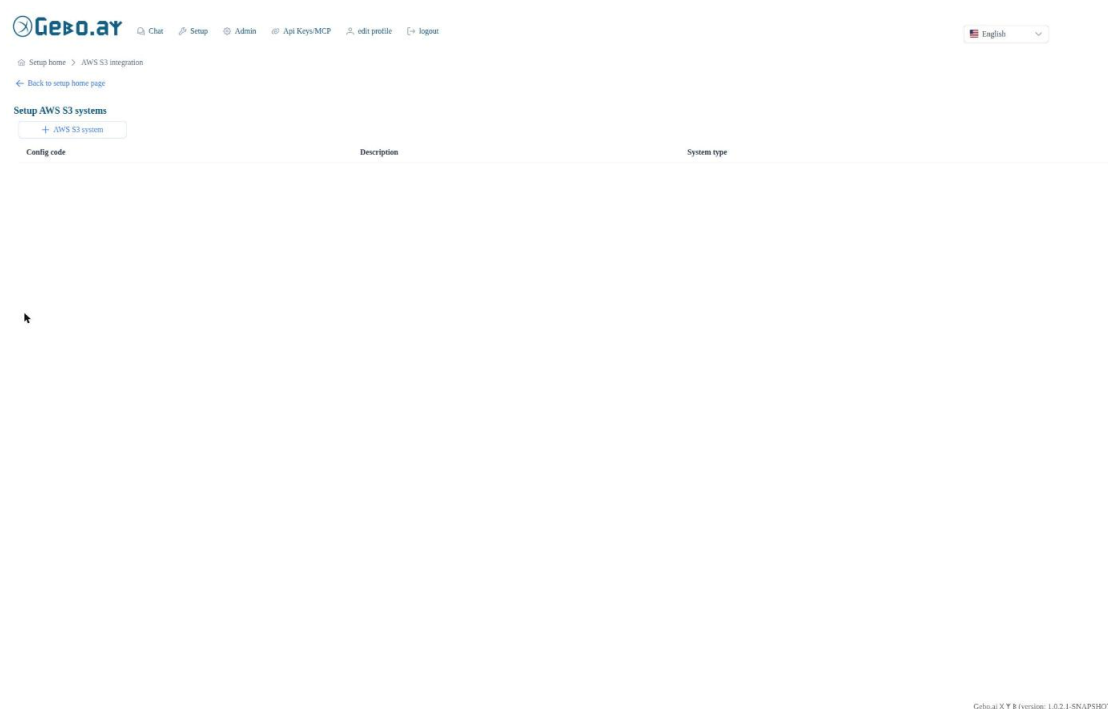


The add form defaults to **Cloud version** and asks for a full Entra ID app registration under "Microsoft oauth2 app registration infos for Gebo.ai": Tenant ID, Client ID, and Secret key. Unlike the SSO module's Microsoft presets documented above, this path goes through Microsoft's own Azure Identity SDK directly rather than Gebo's OAuth2 provider library — it isn't affected by the tenant-substitution bug fixed earlier in this session.

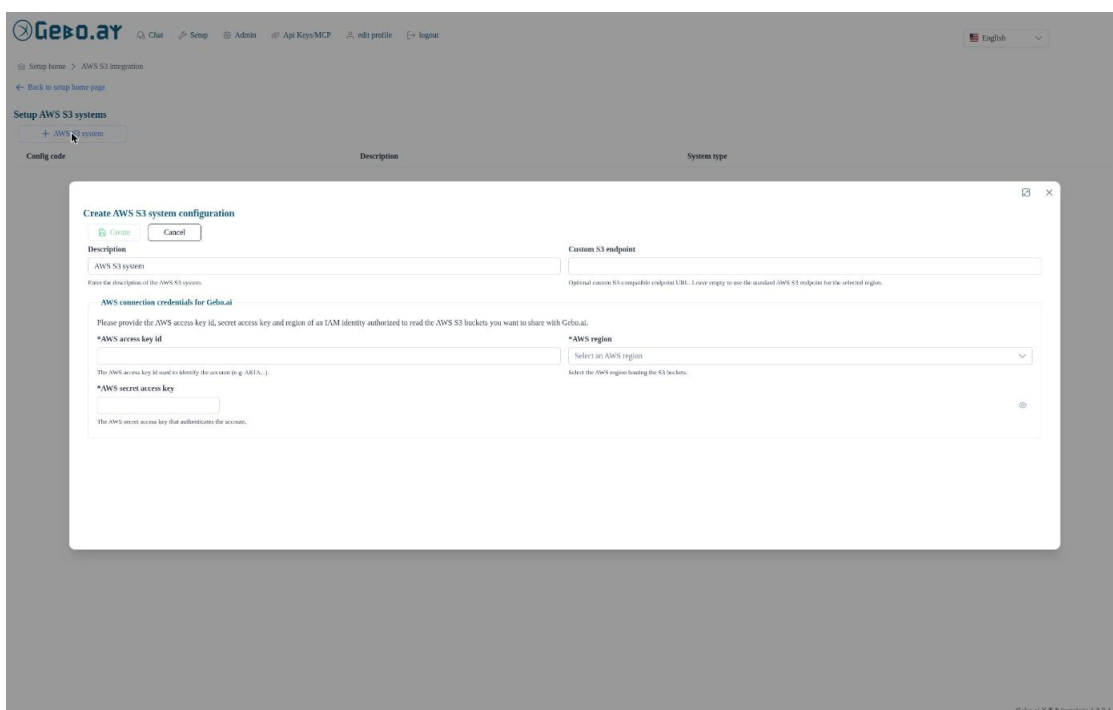
The screenshot shows a modal window titled 'Create SharePoint/OneDrive system configuration'. It has 'Create' and 'Cancel' buttons. The form has two main sections. The first section, 'Description', has a text input for 'Sharepoint/OneDrive system' and a dropdown for '*Sharepoint version' with 'Cloud version' selected. Below this is a small text input for 'Enter the description of the Sharepoint/OneDrive system'. The second section, 'Microsoft oauth2 app registration infos for Gebo.ai', contains instructions: 'Please go to your organization Microsoft Azure Administrative section or Microsoft Entra Administrative section, add a new app for this Gebo.ai installation, note the client-id, tenant-id, and client-secret and report them here'. It has three fields: '*Tenant ID', '*Client ID', and '*Secret key'. Each field has a small text input and a note below it: 'Oauth2 tenant identifier for your organization microsoft "tenant" product (to be retrieved in microsoft cloud admin console)', 'Oauth2 client identifier for this app registered in the microsoft "entra" product (to be retrieved in microsoft cloud admin console)', and 'Oauth2 client secret for this app registered in the microsoft "entra" product (to be retrieved in microsoft cloud admin console)' respectively. At the bottom right of the modal, there's a version string: 'Gebo.ai X.Y.Z (version: 1.0.2.1)'.

AWS S3 integration

Registers AWS S3 (or S3-compatible) bucket sources.

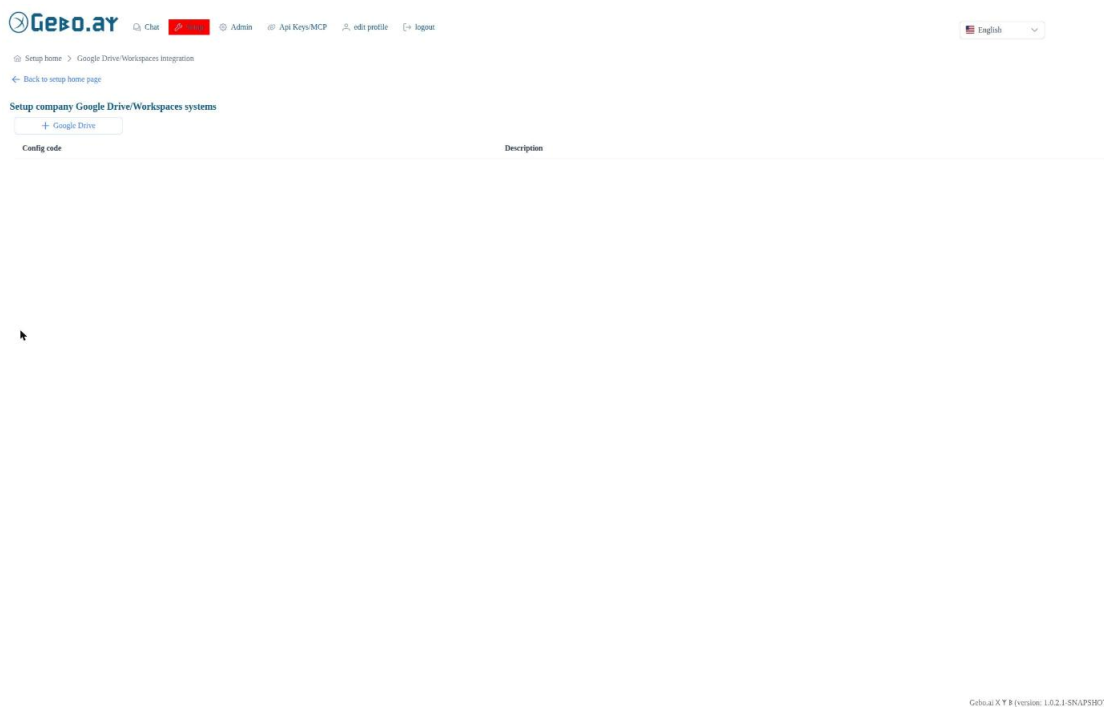


Description, an optional **Custom S3 endpoint** (for S3-compatible object stores — left empty, it uses the standard AWS endpoint for the selected region), then "AWS connection credentials for Gebo.ai": AWS access key id, AWS region (a real region dropdown), and AWS secret access key with a show/hide eye toggle.



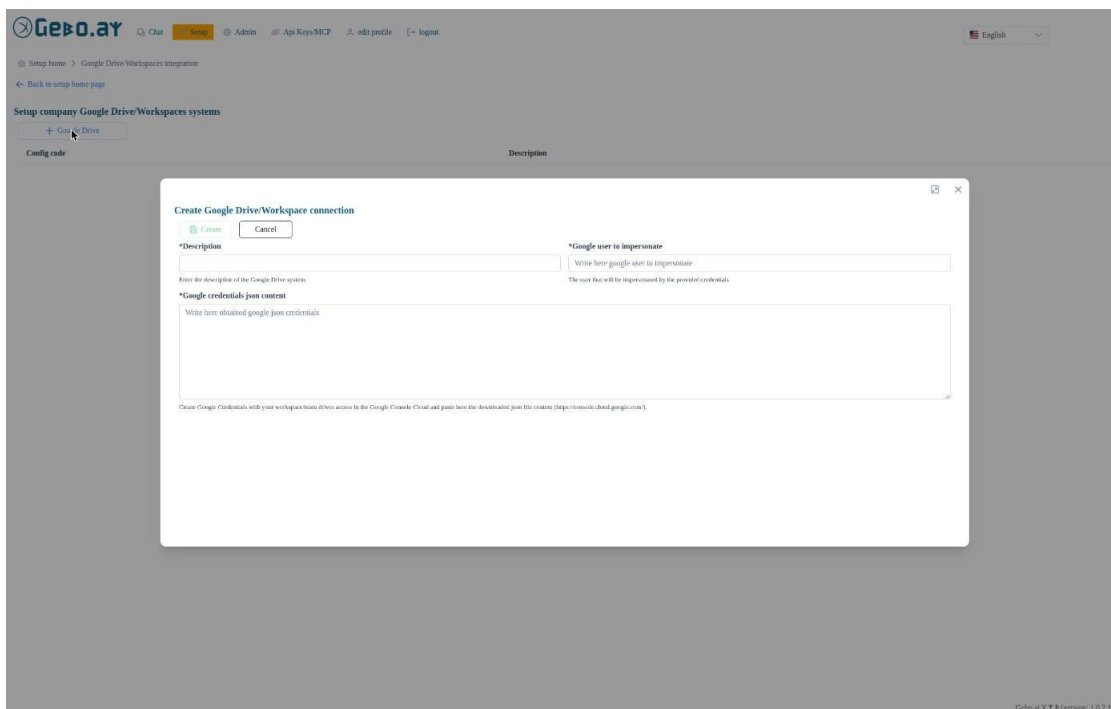
Google Drive/Workspaces integration

Registers Google Drive/Workspace team-data sources.



Gebo.ai X.Y.B (version: 1.0.2.1-SNAPSHOT)

A different shape from every other integration here: Description, a **Google user to impersonate**, and a raw **Google credentials json content** textarea — you create a service account with domain-wide delegation in Google Cloud Console, download its JSON key, and paste the whole file's contents in. No client id/secret pair; the impersonated user plus the service-account JSON is the entire credential.

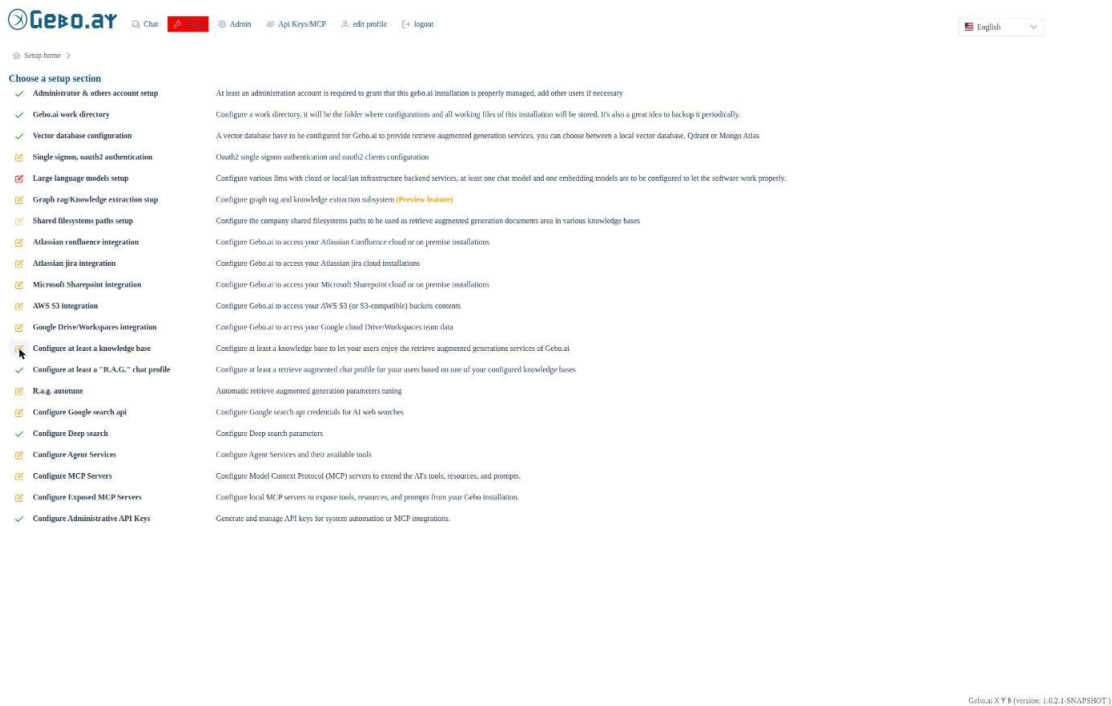


Gebo.ai X.Y.B (version: 1.0.2.1)

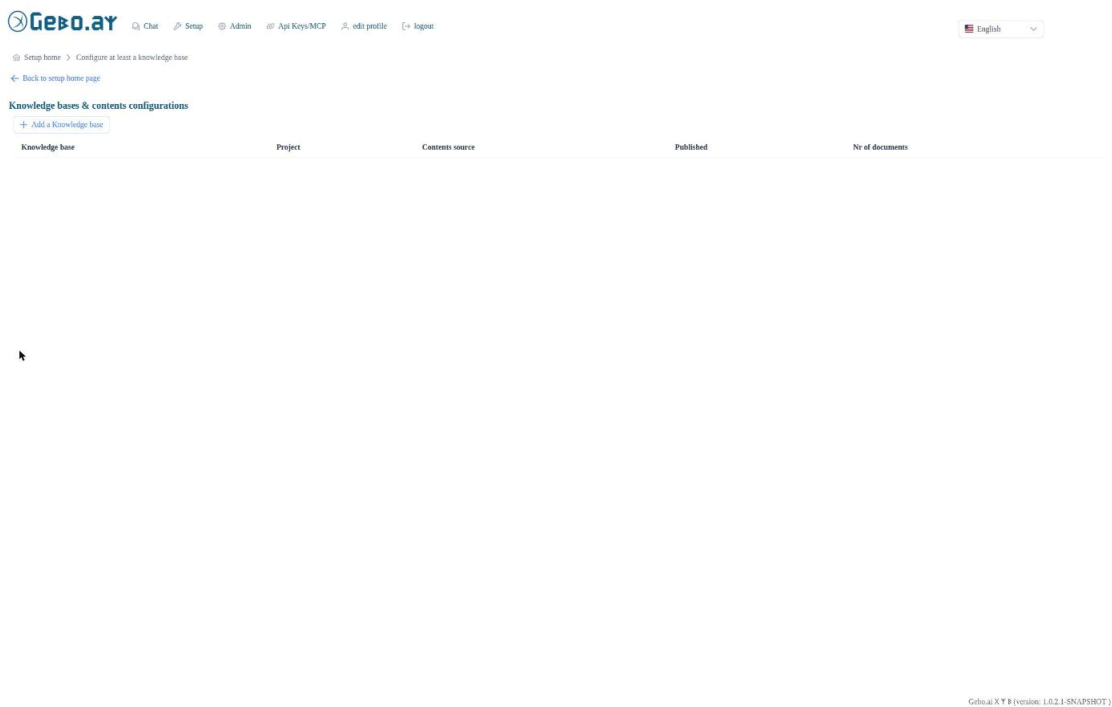
Configure at least a knowledge base

This is where the LLM gating becomes visible in practice: before any embedding model existed, the checklist item simply didn't navigate. After connecting Regolo.ai (Part 2), the same click opens the full knowledge-base management screen and its 5-step creation wizard — create knowledge base, create project, choose data-source type, create data source, publish.

Before an embedding model existed — click has no effect.



After — the knowledge base list opens.



Gebo.ai

Chat

Settings

Admin

API Keys MCP

edit profile

login

English

1

Create new knowledge base

2

Create new project

3

Choose data source type

4

Create new data source

5

Publish data source

Cancel

Forward →

Editing a Knowledge base

General info

Knowledge base access settings

Description

Enter the description of the knowledge base:

Used **embedding models** (leave empty to use system default)

Select embedding models

Embedding models for the knowledge base, leave empty will use the default one

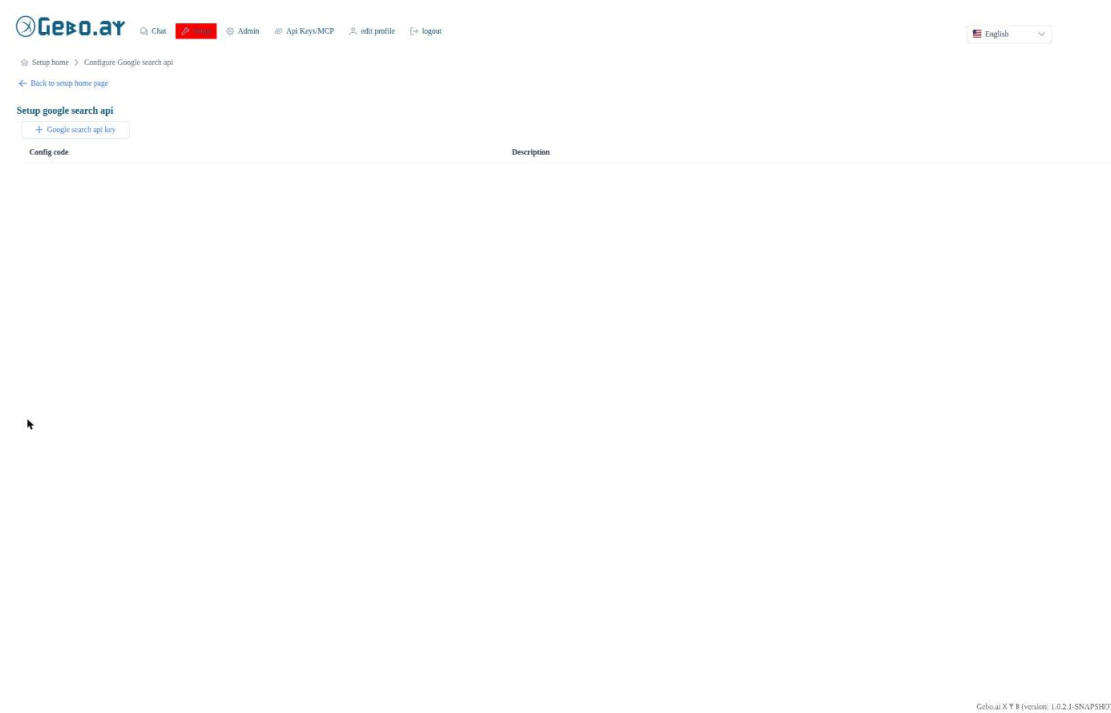
Unique Code

Automatically generated unique code of the knowledge base:

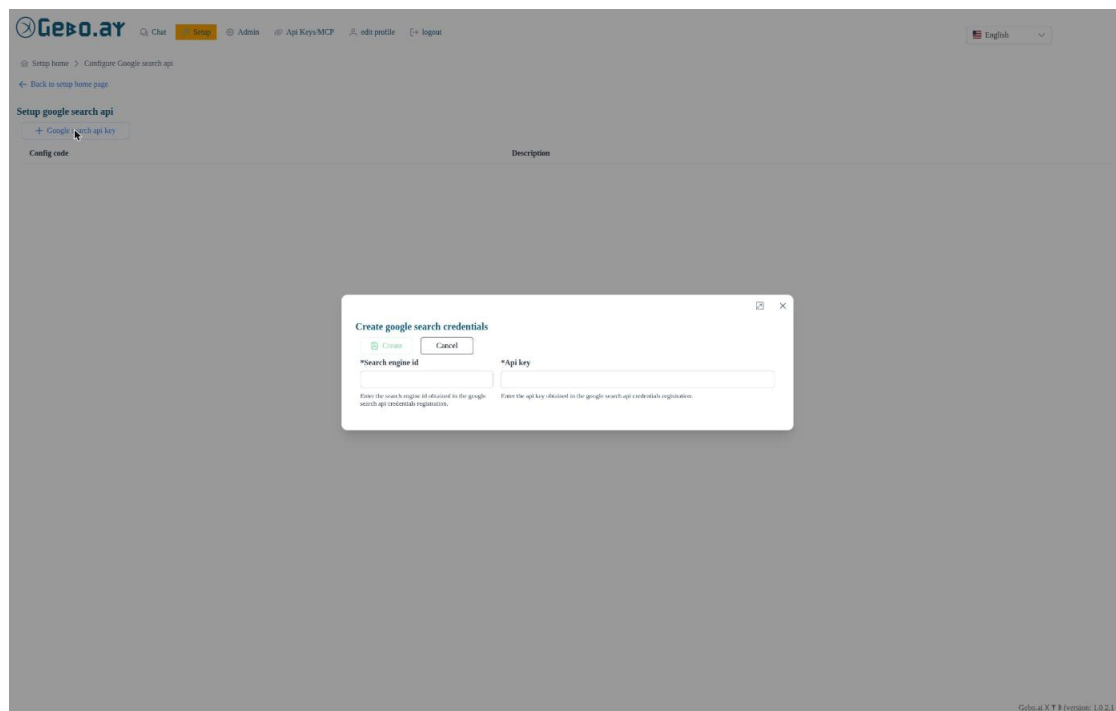
Gebo.ai XT 0 (version: 1.0.21-SNAPSHOT)

Configure Google search API

Registers Google Custom Search credentials, letting chatbots perform live web searches as a tool.



Just two fields: a Google Programmable Search Engine **Search engine id** and its **Api key**, both obtained from Google's search API credentials registration.



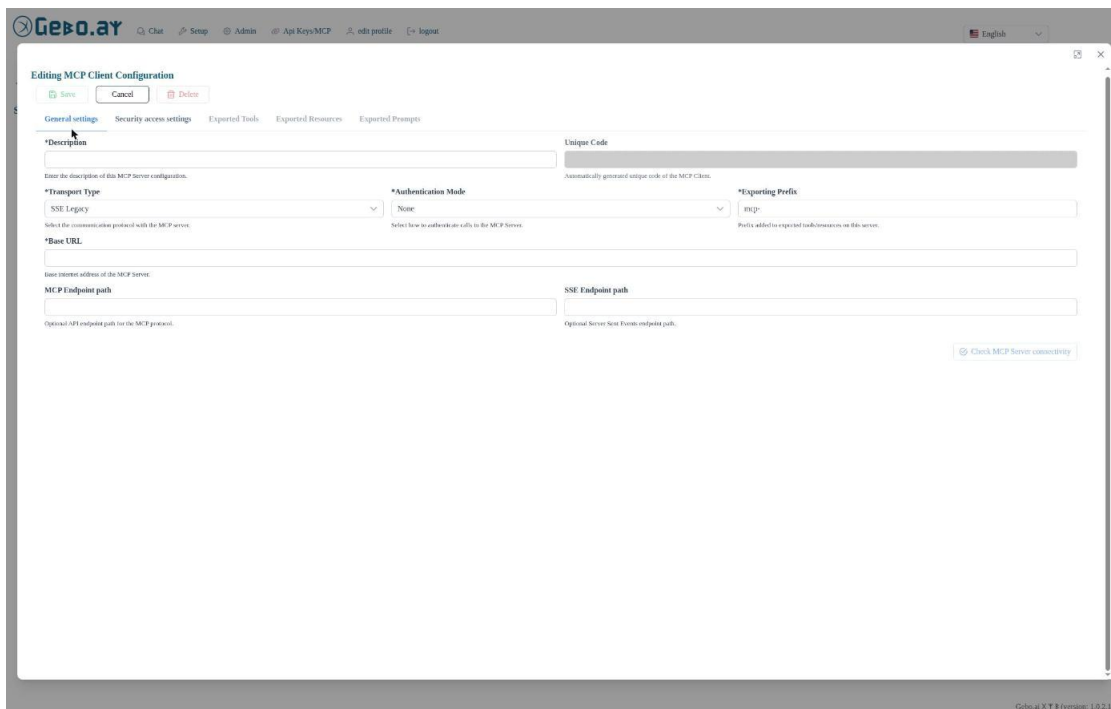
Configure MCP Servers

Connects Gebo.ai to *external* Model Context Protocol servers, giving chatbots extra tools and data sources.

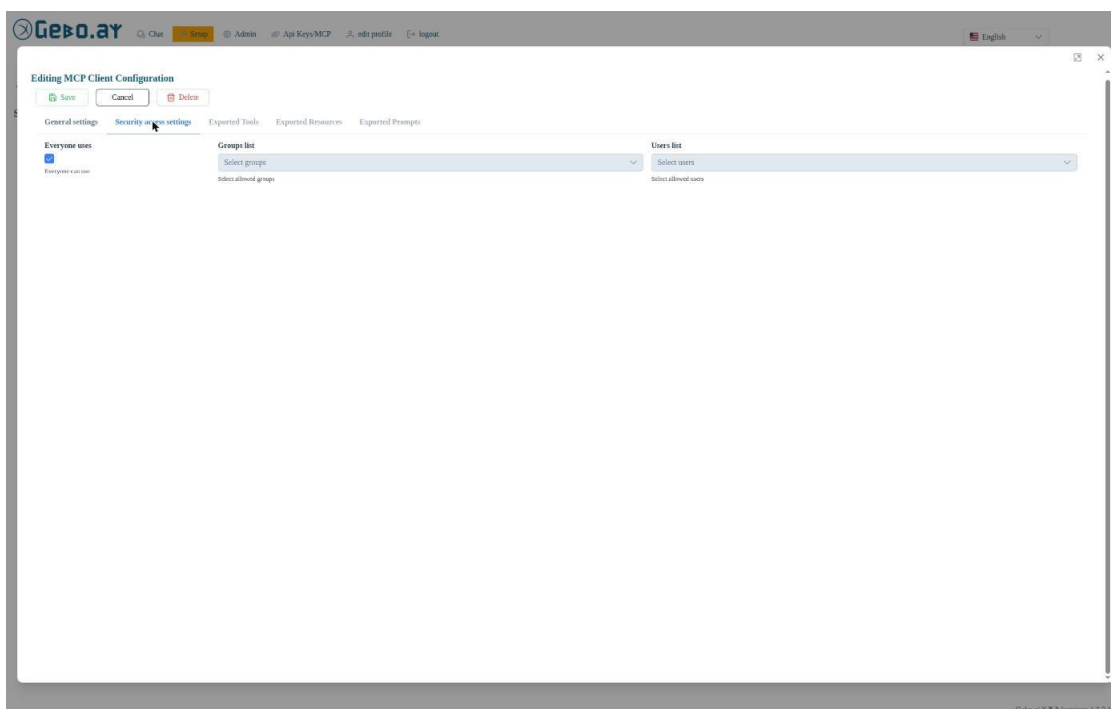


Gebo.ai X.Y.B (version: 1.0.2.1-SNAPSHOT)

"Editing MCP Client Configuration" — General settings tab: Description, auto-generated Unique Code, **Transport Type** ("SSE Legacy" by default), **Authentication Mode** ("None" by default), an Exporting Prefix (defaults to mcp-, prepended to this server's tools/resources to avoid name clashes with other connected MCP servers), Base URL, and optional MCP/SSE endpoint path overrides. A "Check MCP Server connectivity" button (relabeled "Refresh MCP Server exports" after the first save) tests the connection.



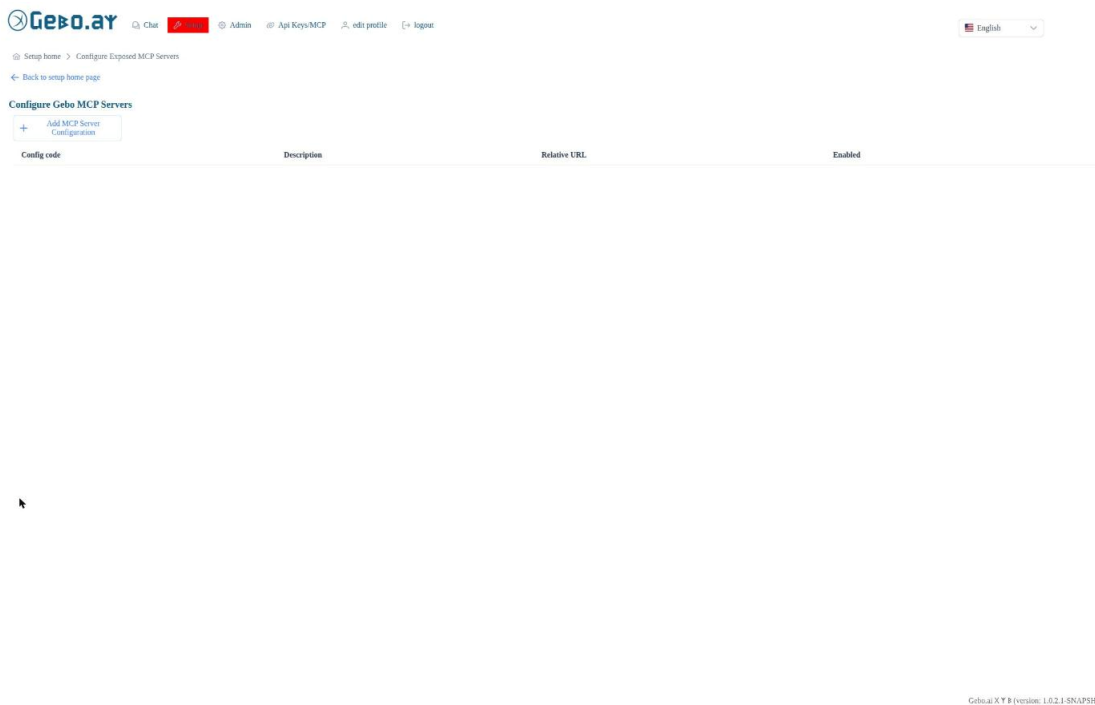
Security access settings tab (unlocks after the first save): the same "Everyone uses" / Groups list / Users list pattern seen throughout this Part.



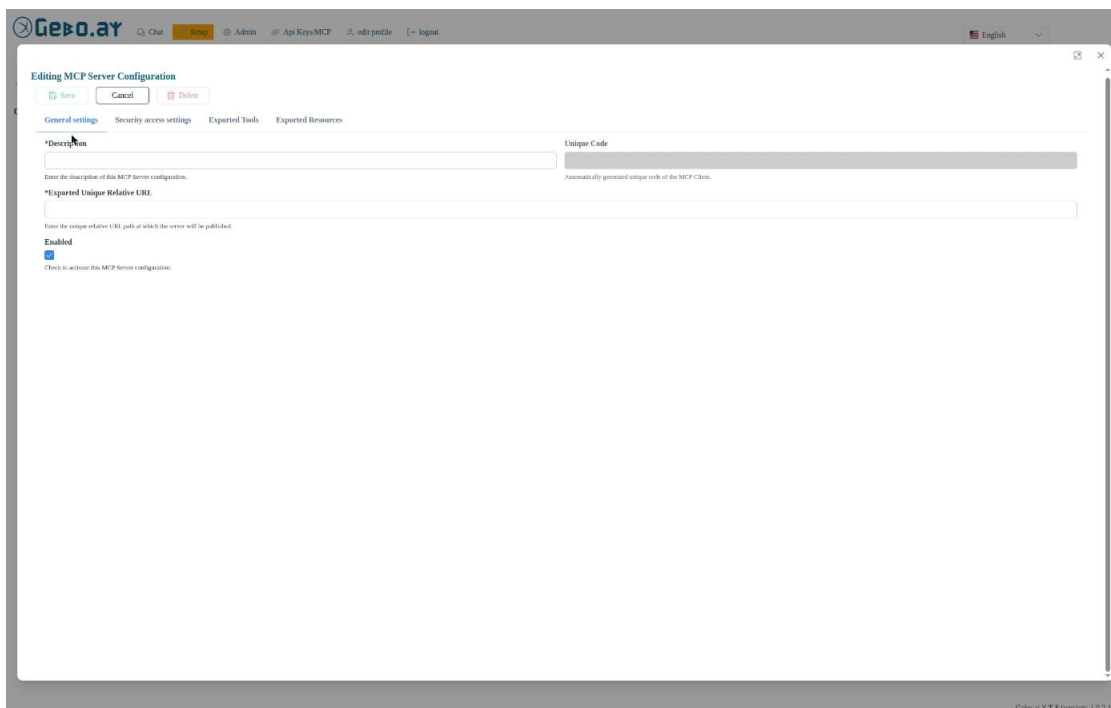
The other three tabs — **Exported Tools**, **Exported Resources**, **Exported Prompts** — stay locked even after saving. They only populate once "Refresh MCP Server exports" succeeds against a real, reachable MCP server, which lists what that server actually offers. A placeholder `http://localhost:9999/mcp` (no server listening) saves fine but leaves these three permanently grayed out — there's no local MCP server in this environment to demonstrate a populated result.

Configure Exposed MCP Servers

The reverse direction: exposes Gebo.ai *itself* as an MCP server that external clients (Claude Desktop, Cursor, IDE extensions) can connect to.

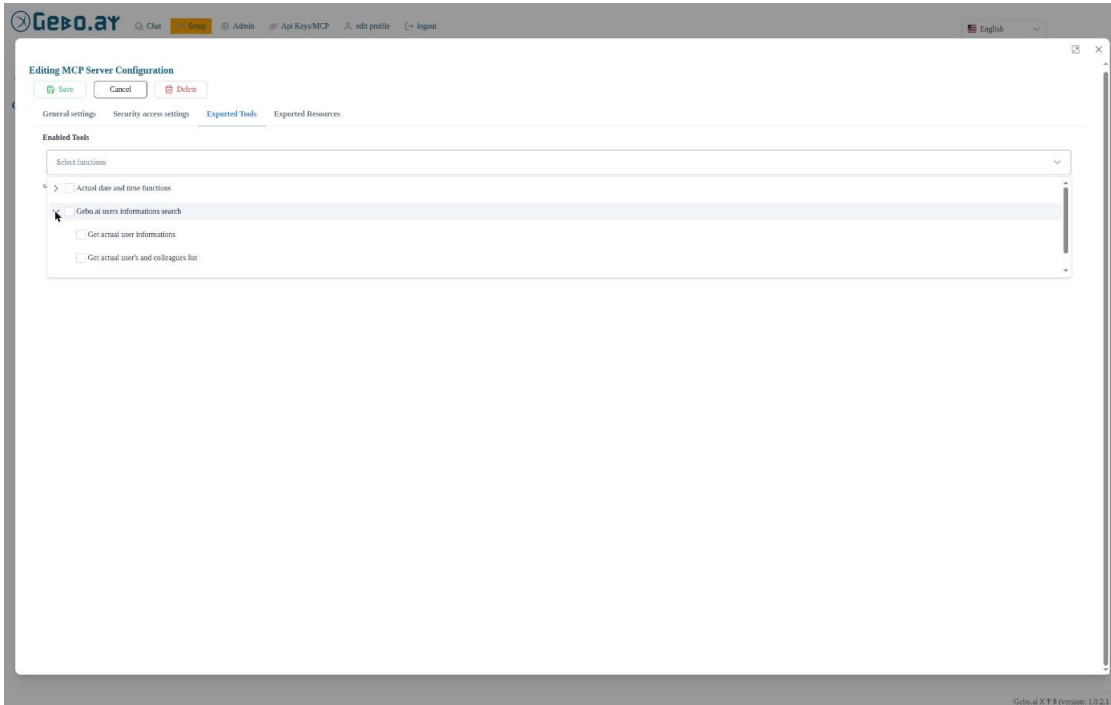


Simpler than the client side: Description, an auto-generated Unique Code, the **Exported Unique Relative URL** path this server will be published at, and an Enabled checkbox (on by default).

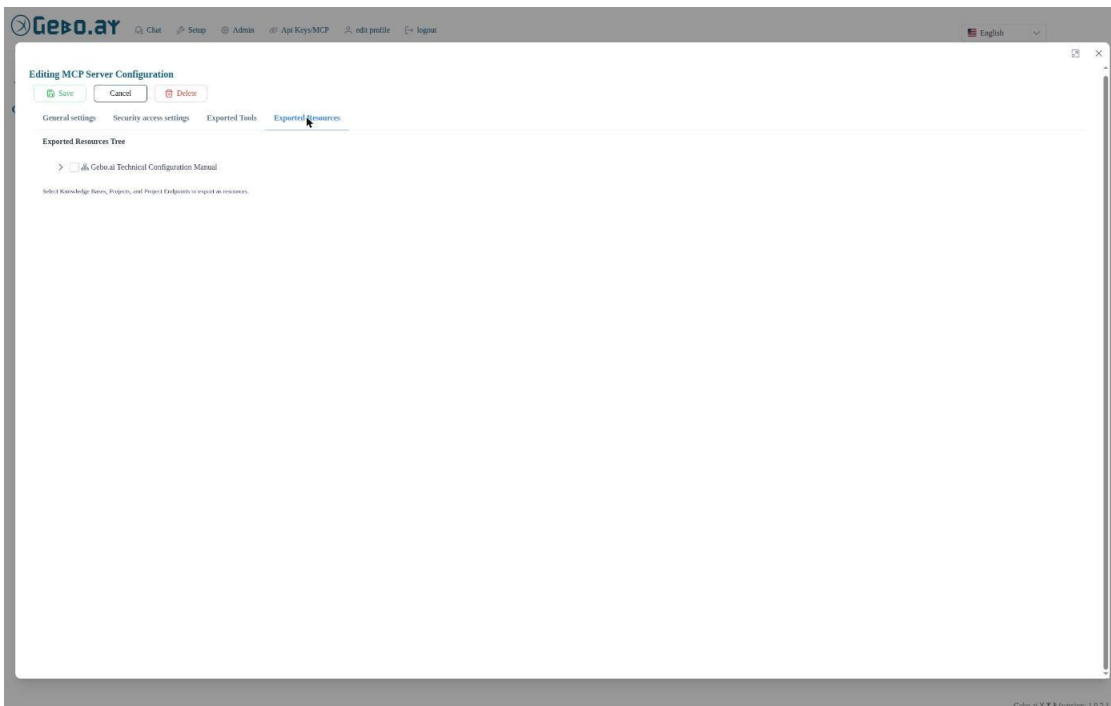


Unlike the client side, all three remaining tabs are usable immediately after the first save, since they draw on Gebo's own capabilities rather than an external server's. **Exported Tools:** a

checkable, nested list of Gebo's own function groups (Actual date and time functions, Gebo.ai users informations search, Internet searching and browsing — each expandable into individual functions like "Get actual user informations") to select exactly what this exposed server hands out.

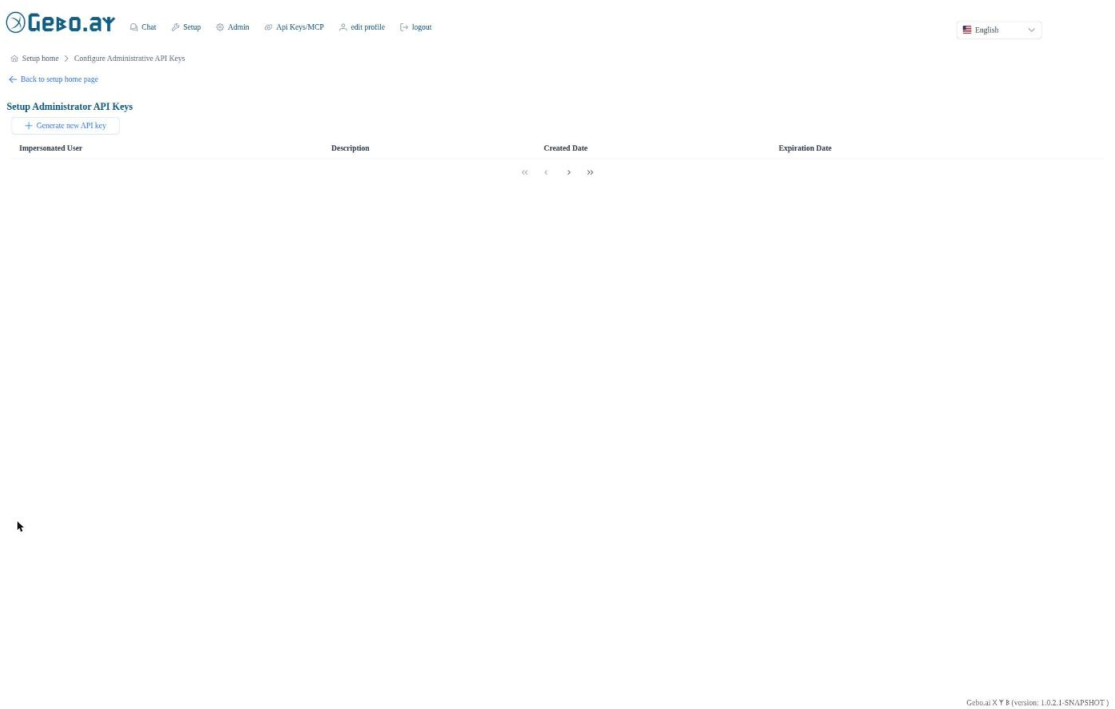


Exported Resources: the same knowledge base/project/data-source tree used throughout this guide, but here for choosing which content gets exposed as MCP resources to external clients. **Security access settings** is identical to the client side's Everyone/Groups/Users pattern.

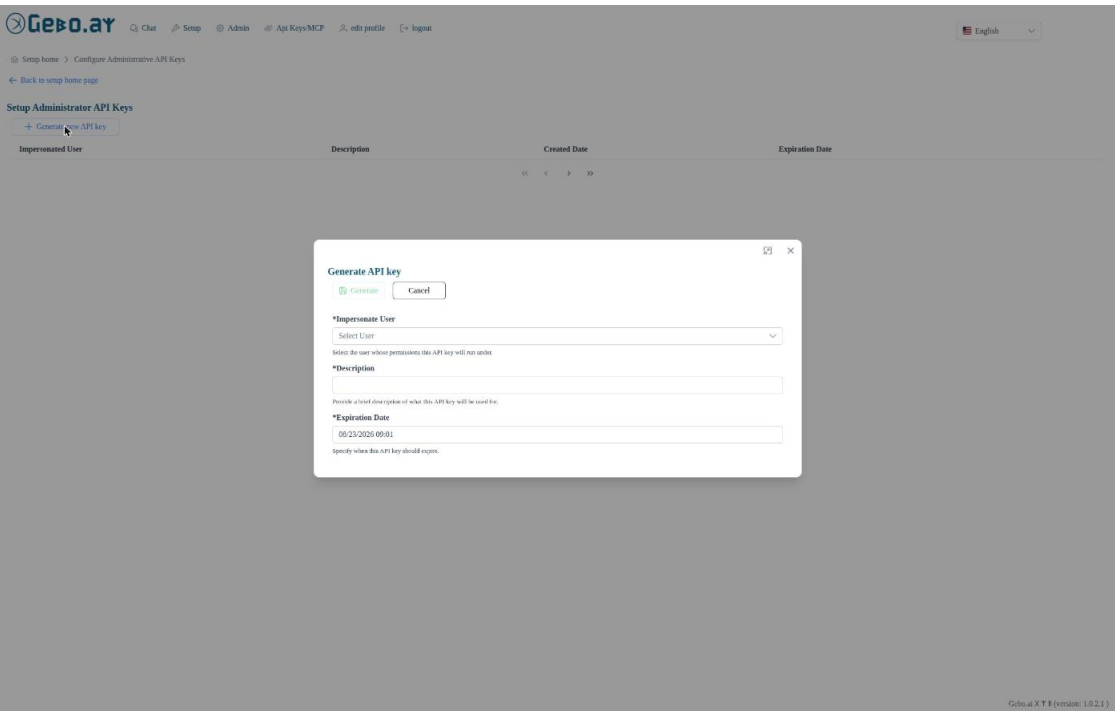


Configure Administrative API Keys

Generates admin-scoped API keys for system automation or MCP integrations, optionally impersonating a specific user, with a configurable expiration date.



"Generate API key": an **Impersonate User** dropdown (the key inherits that user's permissions — there's no separate admin-only scope), a Description, and an Expiration Date pre-filled one year out.

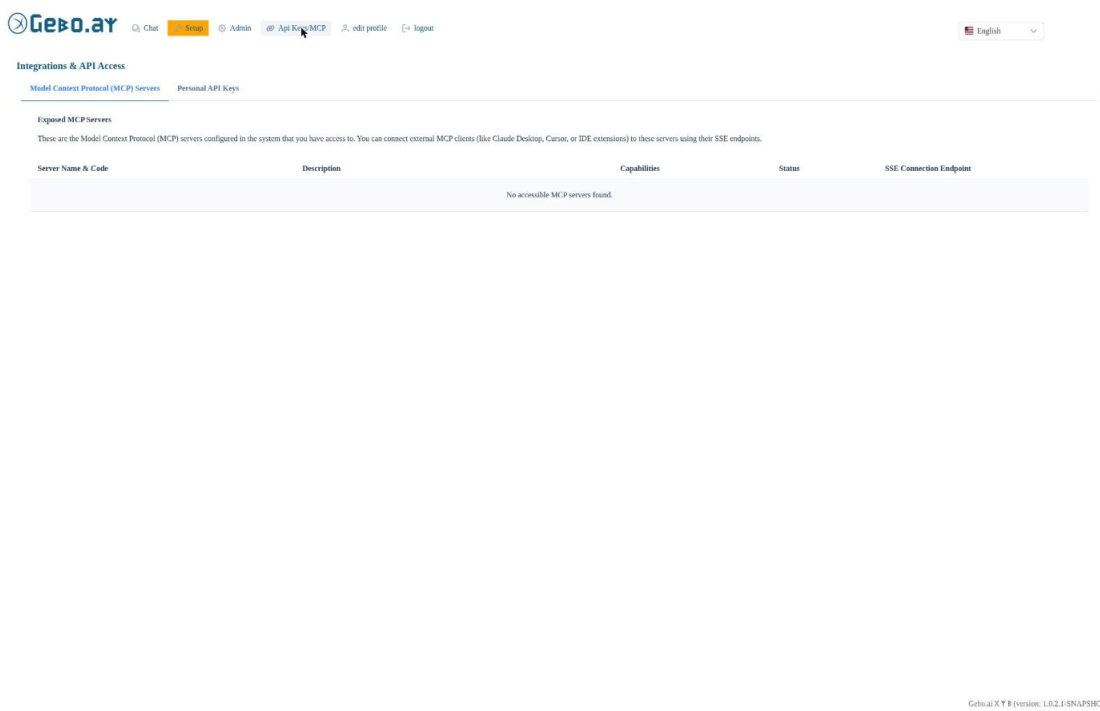


Integrations, API keys & your profile

The two remaining top-navigation items, both scoped to the logged-in user rather than the whole installation.

Api Keys/MCP — Model Context Protocol servers

Lists MCP servers this user has access to, each with its SSE connection endpoint — for connecting external MCP clients like Claude Desktop, Cursor, or IDE extensions to Gebo.ai's exposed capabilities.



Api Keys/MCP — Personal API Keys

Self-service personal API key generation, for authenticating external scripts or MCP clients as this specific user (distinct from the admin-scoped keys in Part 4's "Administrative API Keys" section).



[Chat](#)[Setup](#)[Admin](#)[Api Keys MCP](#)[edit profile](#)[Logout](#)

English

Integrations & API Access

Model Context Protocol (MCP) Servers

Personal API Keys

User API Keys

Generate personal API keys to authenticate your external MCP clients or scripts when connecting to Gebo.ai services.

[Generate New Key](#)

Description	Created At	Expiration Date	Actions
No generated API keys found. Click "Generate New Key" to create one.			

Edit profile

The current user's own account screen — username and a password-change action.



[Chat](#)[Setup](#)[Admin](#)[Api Keys MCP](#)[edit profile](#)[Logout](#)

English

Your user name admin@gebo.ai

[Change password](#)

End-to-end: upload a document & RAG over it

Parts 1–6 documented every screen; this part walks the actual document lifecycle — creating a knowledge base, uploading a real PDF (this project's own `gebo-ai-manual-tech-configuration.pdf` installation manual), publishing it through the ingestion pipeline, and asking the chatbot a question only that PDF could answer. It also documents a real bug hit along the way, and the working alternative path.

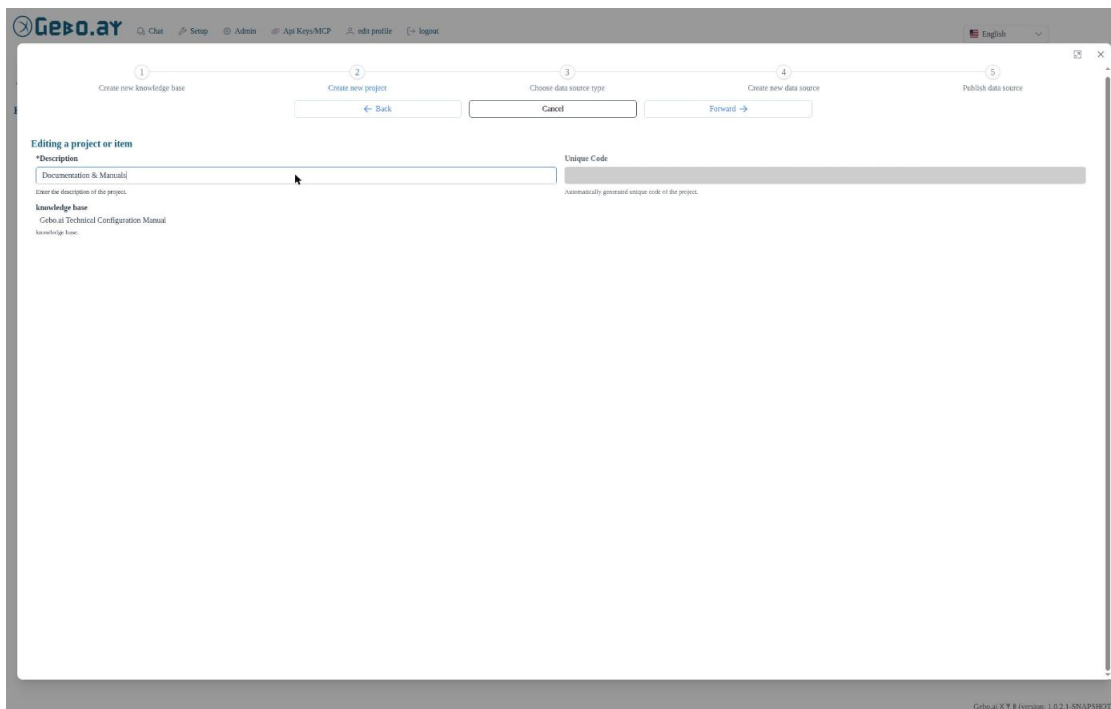
Step 1 — Create the knowledge base

From **Setup** → **Configure at least a knowledge base** → **Add a Knowledge base**, step 1 of the wizard just needs a description.

The screenshot shows the 'Editing a Knowledge base' form in the Gebo.ai application. The form is part of a wizard with five steps: 1. Create new knowledge base, 2. Create new project, 3. Choose data source type, 4. Create new data source, and 5. Publish data source. The current step is 'Editing a Knowledge base', which is divided into 'General info' and 'Knowledge base access settings'. The 'General info' section contains a 'Description' field with the text 'Gebo.ai Technical Configuration Manual', a 'Unique Code' field with a greyed-out value, and a 'Used embedding model' dropdown menu set to 'None empty to use system default'. The 'Knowledge base access settings' section is currently empty. The form is titled 'Editing a Knowledge base' and has a 'Cancel' button and a 'Forward' button.

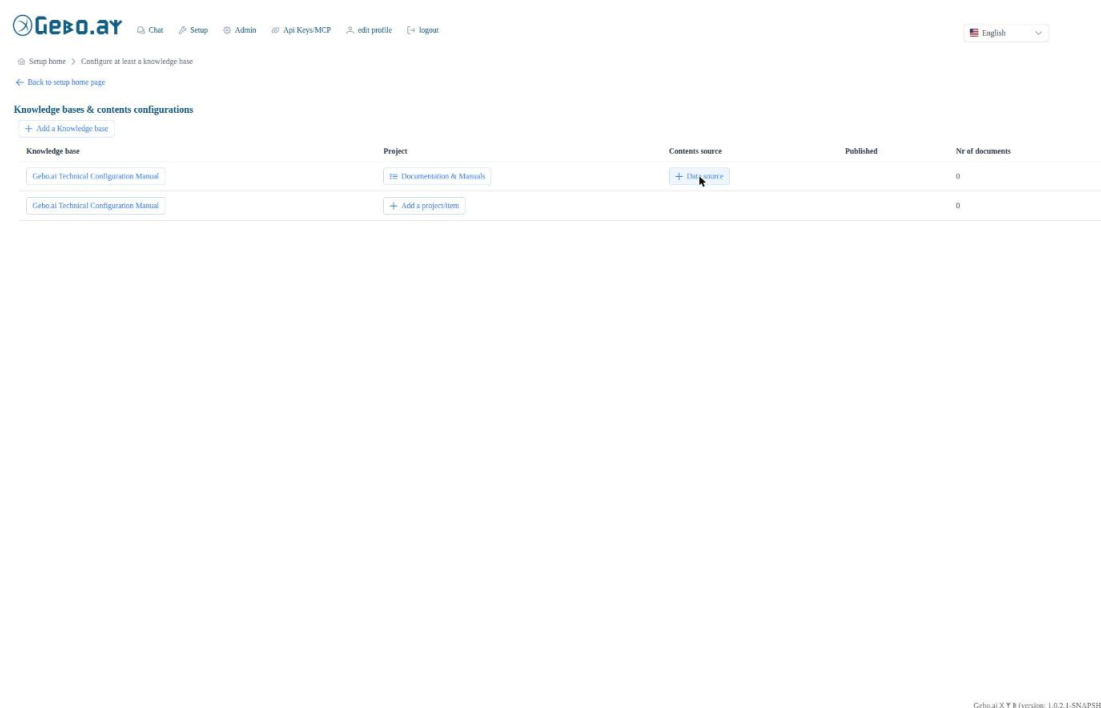
Step 2 — Create a project inside it

Every knowledge base is organized into projects — a grouping unit for related data sources.



A real bug: the quick-add "Data source" button doesn't work

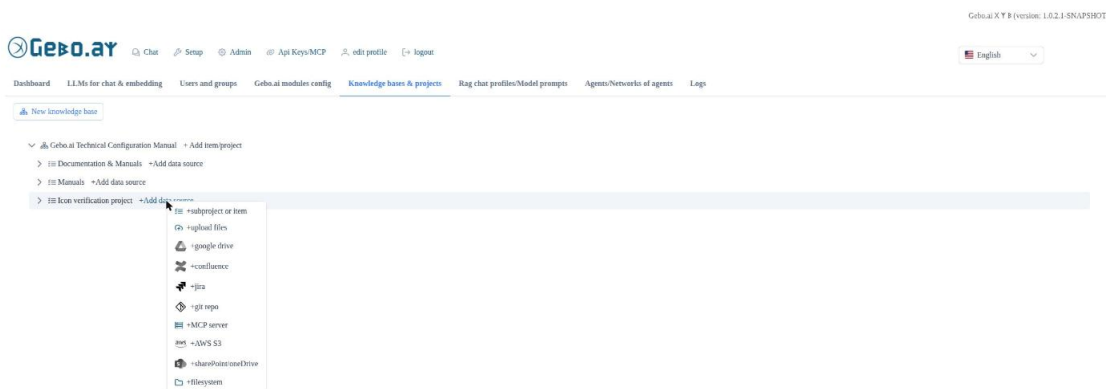
The setup checklist's knowledge-base list has an inline "+ Data source" icon button per project (below). It's not disabled, it's enabled and clickable — but clicking it, via mouse, via accessibility-tree reference, and via a direct JavaScript element.click(), produces **zero effect**: no navigation, no HTTP request, nothing. This was confirmed with network-request monitoring showing no call fired at all. It appears to be a genuine defect in this build (1.0.2.1-SNAPSHOT), not a fluke.



Workaround: use **Admin → Knowledge bases & projects** instead. Its tree view exposes the same action as a text link ("Add data source") next to each project, and that one works correctly.

The working path — Admin's knowledge base tree

Once a project has been saved via this screen, its tree node reliably expands to show a functioning "+Add data source" link, unlike the setup checklist's broken shortcut. Clicking it opens a full menu of data-source types: subproject, upload files, Google Drive, Confluence, Jira, Git repo, MCP server, AWS S3, SharePoint/OneDrive, and local filesystem — the same connector set documented throughout Part 4.



Screenshot reflects the current build: the AWS S3 icon here previously rendered at over double the size of its siblings and in color instead of grayscale (it reused the OAuth2 picker's larger .pi-amazon CSS class). Fixed by giving it its own .pi-aws-s3 class sized like the rest of the menu — verified live in this exact screen.

Step 3 — Upload the file

Choosing "upload files" opens the actual upload form: a description, the parent project (pre-filled), an "Upload contents" file picker, an "Open zip files" toggle for archives, and an optional build-system inspector for source-code uploads.

The image displays two screenshots of the Gebo.ai web interface, specifically the "Editing uploaded content" form. The top screenshot shows the form with empty fields, while the bottom screenshot shows the form with pre-filled data.

Top Screenshot (Empty Form):

- Description:** A text input field with a placeholder "Enter the description of the uploaded content of this system entry."
- Project:** A dropdown menu with "Documentation & Manuals" selected.
- Upload contents:** A file picker button labeled "Upload contents".
- Open zip files:** A toggle switch labeled "Enable zip files opening and content extraction".
- Configure build system:** A section with a "+" button and a list of build systems.

Bottom Screenshot (Filled Form):

- Description:** "Gebo.ai Technical Configuration Manual (PDF)"
- Unique Code:** "documentation-manuals-gebo-ai-technical-configuration-manual-pdf"
- Project:** "Documentation & Manuals"
- Upload contents:** "gebo-ai manual-technical-configuration.pdf"
- Open zip files:** The toggle switch is turned on.
- Configure build system:** The same section with a "+" button and a list of build systems.

After selecting the file, the "Upload contents" field lists gebo-ai-manual-tech-configuration.pdf by name — confirmation the browser actually transferred it, not just that a filename was typed.

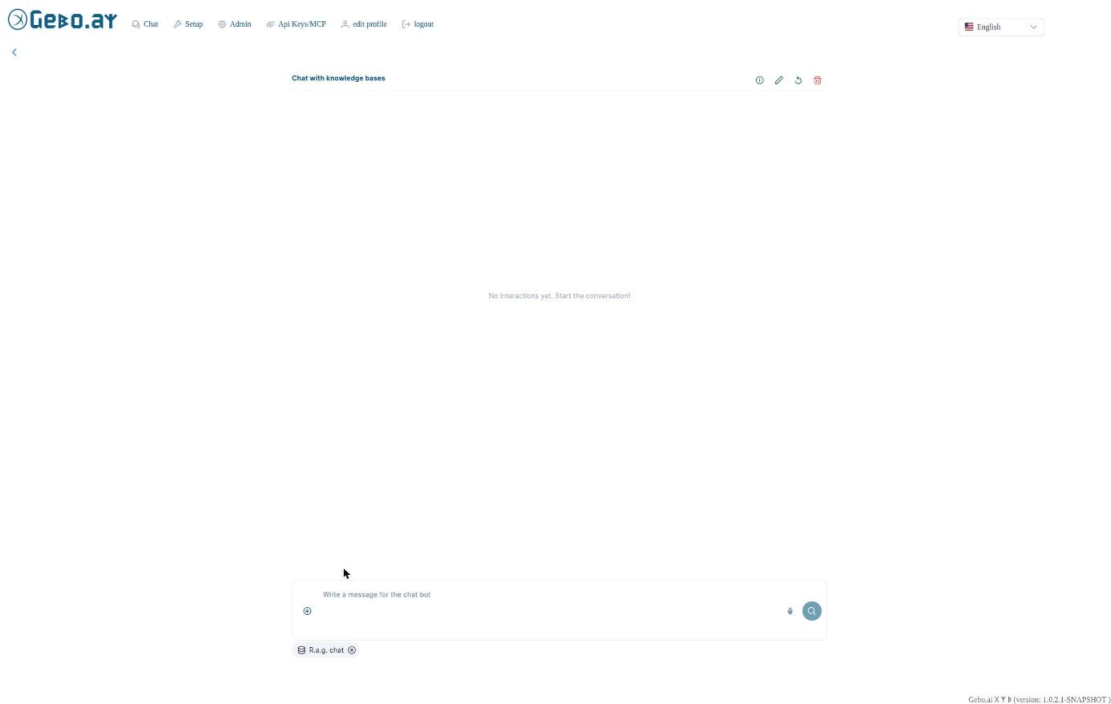
Step 4 — Publish

Clicking **Publish!** immediately opens a live publishing log with per-stage progress: analyzing the document list, tokenization, semantic indexing (the embedding step that populates Qdrant), and full-text indexing (OpenSearch) — each with its own completion pie chart and a token-throughput bar chart. Every stage finished "Processed completely" with zero errors.



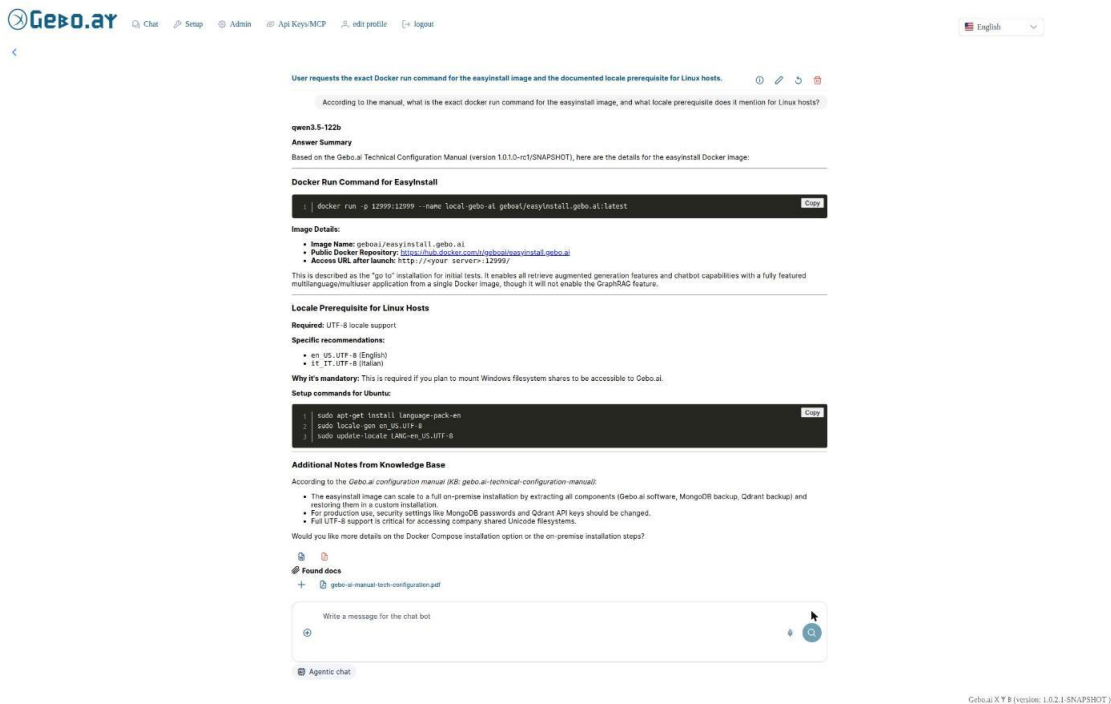
Step 5 — Ask the chatbot about it

Back in Chat, the "+" attachment menu next to the message box exposes a mode switcher — Upload file(s), Browse/search company files, Agentic chat, **R.a.g. chat**, Knowledge base deep search, and Image generation. Selecting R.a.g. chat scopes the next message to retrieval over the configured knowledge base(s).



The result: a grounded answer with a citation

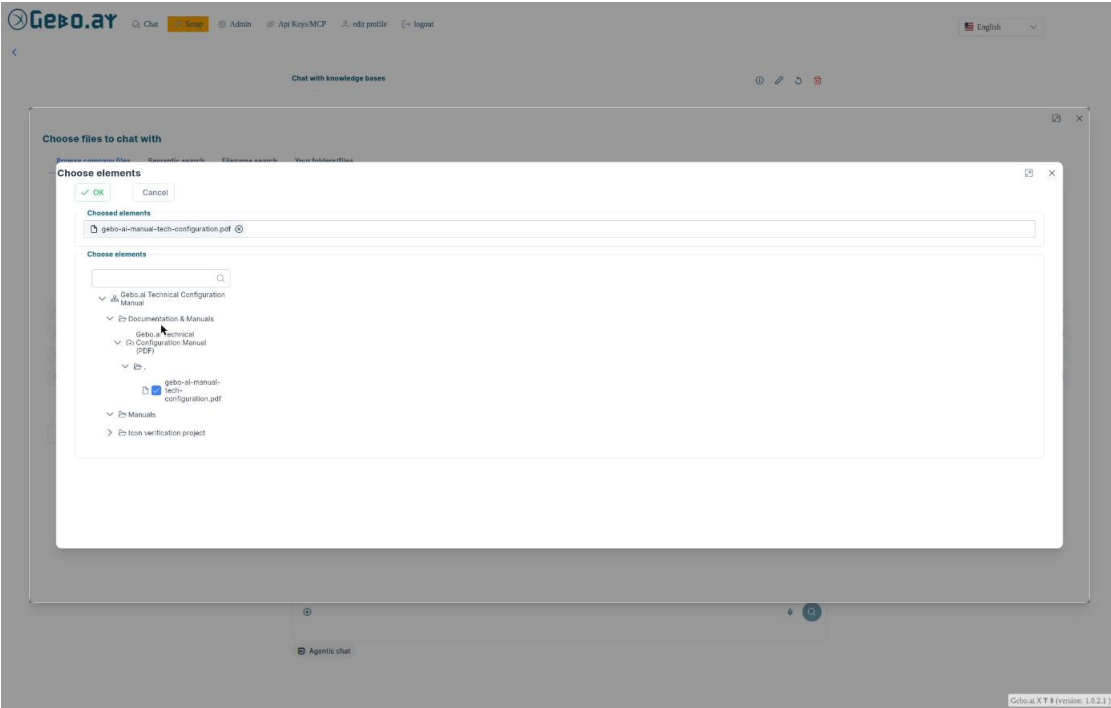
Asked for the exact docker run command for the easyinstall image and its Linux locale prerequisite — details that only exist in the uploaded PDF, not in any LLM's training data — qwen3.5-122b returned the precise command (`docker run -p 12999:12999 --name local-gebo-ai geboai/easyinstall.gebo.ai:latest`), the exact `en_US.UTF-8` / `it_IT.UTF-8` locale requirement, and the matching `apt-get/locale-gen/update-locale` commands — word-for-word consistent with the source document. The response ends with a "📄 **Found docs: gebo-ai-manual-tech-configuration.pdf**" citation, and an "Additional Notes from Knowledge Base" section explicitly attributed to *Gebo.ai configuration manual (KB: gebo.ai-technical-configuration-manual)*.



End-to-end proof the pipeline works: upload → chunking → embedding → indexing → retrieval → grounded, cited generation, using the project's own real documentation as the test corpus.

Reaching an actual file through "Browse company files"

The "Browse company files" tab's tree isn't KB → Project → file. Each data source under a project renders as its own node (labeled with the data source's description, not the word "folder") — that node is a **virtual folder**, not a file. Expanding it reveals the ingested content's real folder structure, starting from a root node literally named `..`. Individual files appear as leaves further down, each with a checkbox to select it for the chat.

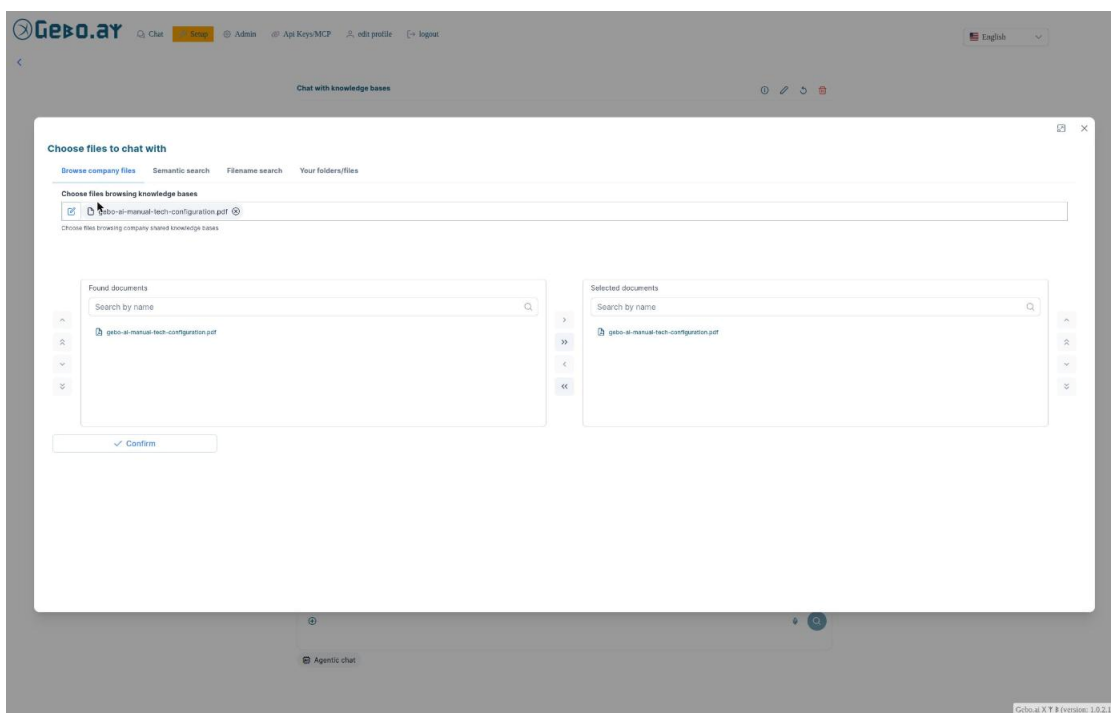


Full path for this install's uploaded manual: **Gebo.ai Technical Configuration Manual** (knowledge base) → **Documentation & Manuals** (project) → **Gebo.ai Technical Configuration Manual (PDF)** (the data source — despite the label, this is the virtual folder, not the file) → . (root folder) → gebo-ai-manual-tech-configuration.pdf (the actual file, checked to select it).

The tree's expand toggles were unreliable via automated clicks during this walkthrough — clicking a chevron often did nothing on the first attempt and needed a retry, or in this specific case needed a manual click from the person driving the browser session rather than the automation. If a node looks stuck collapsed, retry the click before assuming there's nothing underneath.

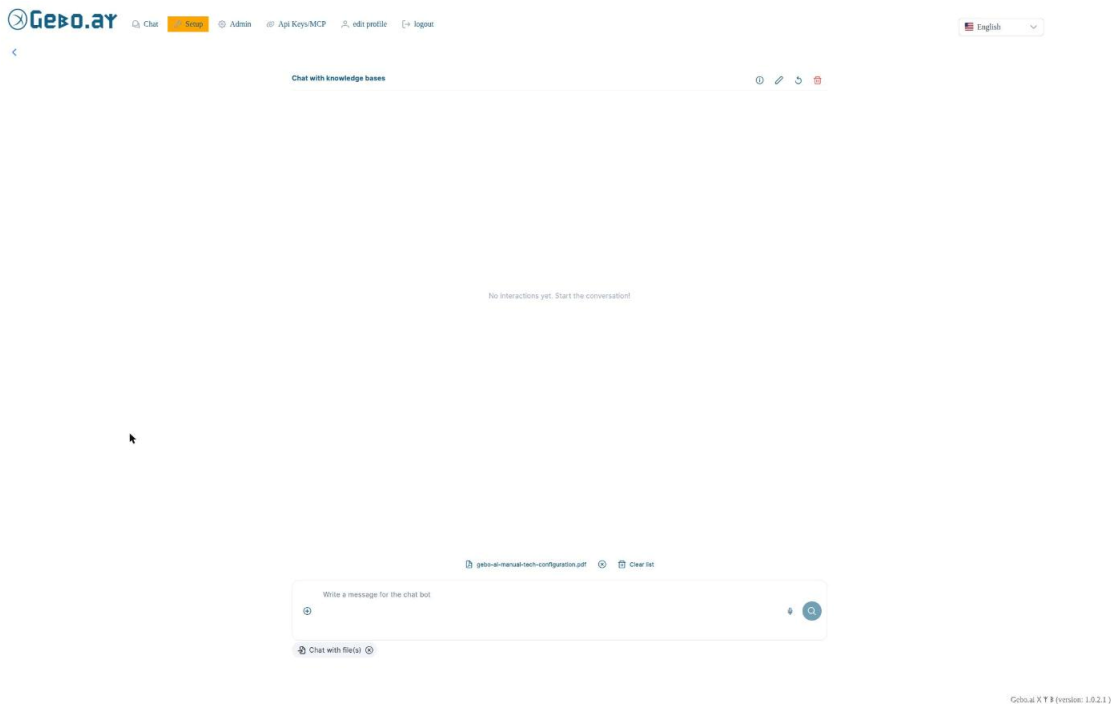
Back in "Choose files to chat with"

Clicking "OK" on the tree dialog returns to the tabbed picker, with the file now populated in both "Found documents" and "Selected documents".



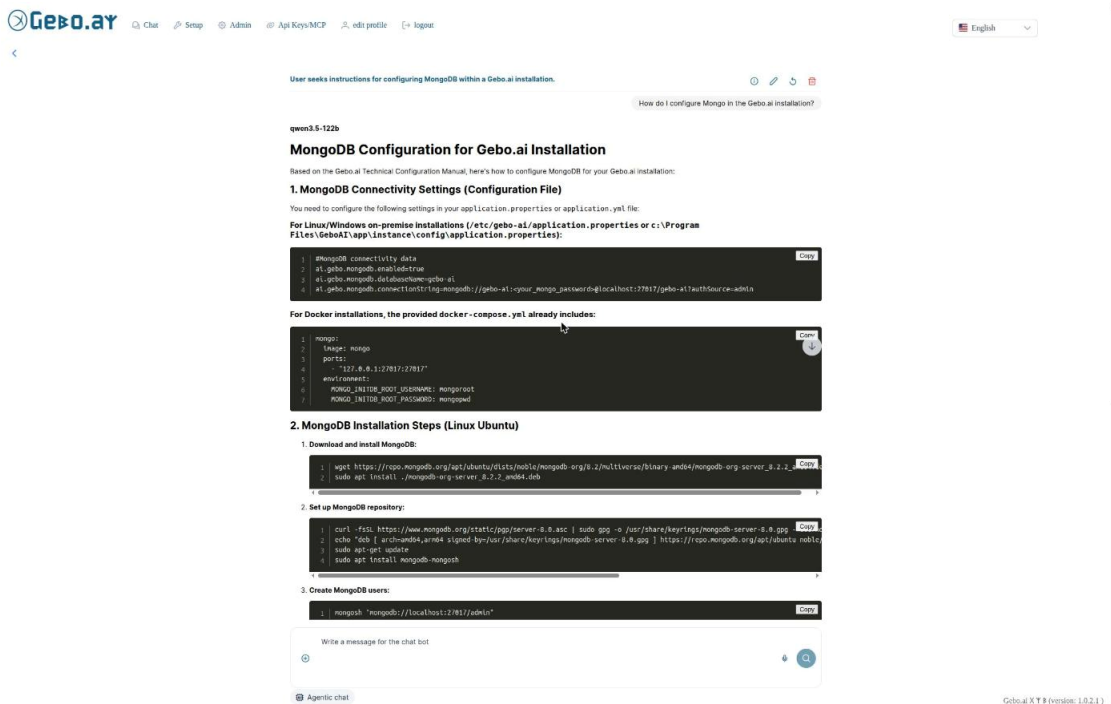
Back in Chat, attached and ready

"Confirm" returns to the chat window with the file pinned above the message box and the mode chip switched from "Agentic chat" to **"Chat with file(s)"** — the next message is now scoped specifically to this document, distinct from the R.a.g. chat mode used earlier in this Part.



Proof: "Chat with file(s)" is genuinely scoped to the attached document

Asked "How do I configure Mongo in the Gebo.ai installation?" with only gebo-ai-manual-tech-configuration.pdf attached. The answer returns exact application.properties keys (ai.gebo.mongodb.enabled, ai.gebo.mongodb.databaseName, the connection string), the Docker Compose mongo: service block, and step-by-step Ubuntu install commands — none of it generic MongoDB knowledge, all of it specific to this document.



The response closes with a "Sources Used" section naming the exact file and page numbers ("MongoDB connectivity settings", page 28; "Install mongodb community edition", page 9; etc.)

and a "Found docs" footer citing the same PDF — this is retrieval grounded in the attached file, not the model's general training knowledge.

Gebo.ai is an open-source, vendor-agnostic enterprise RAG & agentic AI platform. See gebo.ai and the project's README.md / docs/APPLICATION-YML-ADMIN-MANUAL.md for architecture and configuration reference alongside this visual guide.

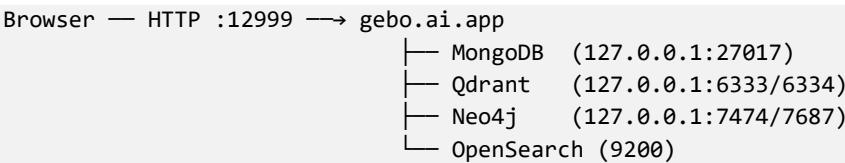
Gebo.ai hardening guide, architecture overview

Gebo.ai runs as a **Spring Boot** application and supports two deployment architectures:

Monolithic (`gebo.ai.app`)

A single Spring Boot process (JAR / Docker image `geboai/gebo.ai`) that bundles all modules.

It listens on **port 12999** (HTTP) and connects to co-located infrastructure databases.

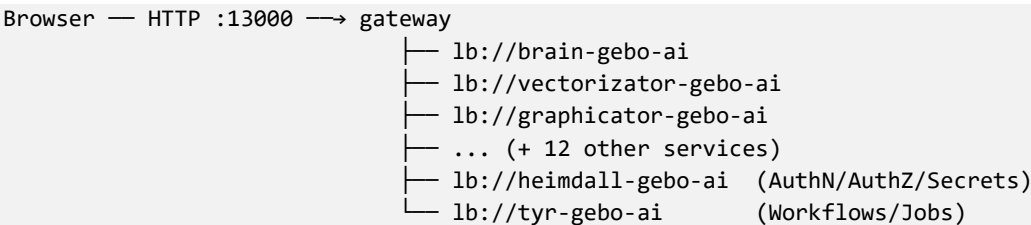


Docker Compose: `dockers/gebo.ai/docker-compose.yml`

Microservices (19 distinct services)

Each module runs as its own Spring Boot service on a dedicated port (13000–13019).

The **gateway** (port 13000) is the single entry point for browsers. All internal service-to-service calls resolve through **Eureka** (service registry, port 13017) with client-side load balancing.



Docker Compose: `dockers/gebo.microservices/docker-compose.yml`

Shared Infrastructure (both deployments)

Component	Default Ports	Internal Auth
MongoDB	27017	`mongoroot / mongopwd`
Qdrant	6333, 6334	API key `ce7c85bc-...`

Neo4j	7474 (HTTP), 7687 (Bolt)	`neo4j / neo4jmaster`
OpenSearch	9200, 9600	`admin / dothesearch1973-Advanced`
RabbitMQ (microservices only)	5672, 15672 (mgmt)	`guest / guest`

⚠ All default credentials are shipped as placeholder values and must be changed

before production deployment.** See [Part C — Secrets Rotation Checklist](#4-part-c--secrets-rotation-checklist).

2. Part A — Enabling HTTPS (SSL/TLS)

Gebo.ai is a standard **Spring Boot 3.x** application. It currently listens on

plain HTTP. Enabling HTTPS is a standard Spring Boot `server.ssl.*` configuration.

The same approach applies to every service in the stack because they all share the same embedded web server (Tomcat or Netty/WebFlux).

Spring Boot Reference:

[Configure SSL](https://docs.spring.io/spring-boot/reference/web/graceful-shutdown.html#web.ssl)

in the official Spring Boot documentation.

2.1 Monolithic Deployment

The monolith exposes **port 12999** on all host interfaces. You harden it by:

1. generating a keystore,
2. mounting it into the container,
3. adding `server.ssl.*` properties to the config file.

Step 1 — Generate a PKCS12 Keystore

Use the Java `keytool` (bundled with any JDK) or `openssl`:

```
# Using keytool (recommended for Spring Boot)
keytool -genkeypair \
  -alias gebo-ssl \
  -keyalg RSA \
  -keysize 2048 \
  -storetype PKCS12 \
  -keystore /home/gebo.ai/secrets/gebo-ssl.p12 \
  -validity 3650 \
  -dn "CN=gebo.example.com, OU=IT, O=MyOrg, L=City, S=State, C=IT"
```

You will be prompted for a keystore password and a key password. Keep these secure.

For production, obtain a certificate signed by a trusted CA (Let's Encrypt, DigiCert, etc.) and import it into the keystore instead of using a self-signed one.

See [Section 2.3](#23-certificate-generation-reference) for Let's Encrypt integration.

Step 2 — Mount the Keystore in Docker

Edit `dockers/gebo.ai/docker-compose.yml` and add a volume mount to the `gebo.ai`

service:

```
gebo.ai:
  image: geboai/gebo.ai
  restart: always
  ports:
    - "12999:12999"          # ← change to HTTPS port, e.g. "8443:12999"
  volumes:
    - /home/gebo.ai/work:/opt/gebo.ai/work
    - /home/gebo.ai/home:/opt/gebo.ai/home
    - /home/gebo.ai/logs:/opt/gebo.ai/logs
    - /mnt:/opt/gebo.ai/shares
    - /home/gebo.ai/secrets/gebo-ssl.p12:/opt/gebo.ai/secrets/gebo-ssl.p12:ro # ← NEW
```

Step 3 — Add SSL Properties

Edit `dockers/gebo.ai/config/application.yml`. Replace the existing `server:` block:

```
server:
  port: 12999
  ssl:
    enabled: true
    key-store-type: PKCS12
    key-store: file:/opt/gebo.ai/secrets/gebo-ssl.p12
    key-store-password: ${GEBO_SSL_KEYSTORE_PASSWORD}
    key-alias: gebo-ssl
  compression:
    enabled: true
    mime-types: text/html,text/xml,text/plain,text/css,text/javascript,...
    min-response-size: 1024
  http2:
    enabled: true
```

The password is injected via an environment variable so it never sits in plain text

in the config file. Set it in the `environment:` block of the compose file:

```
gebo.ai:
  environment:
    GEBO_SSL_KEYSTORE_PASSWORD: "your-strong-password-here"
```

Step 4 — Redirect HTTP to HTTPS (Optional)

To redirect `http://gebo.example.com:12999` → `https://gebo.example.com:12999`,

add a second embedded connector. In the main application class or a

`@Configuration` class, add:

```
@Bean
public ServletWebServerFactory servletContainer() {
    TomcatServletWebServerFactory tomcat = new TomcatServletWebServerFactory() {
        @Override
        protected void postProcessContext(Context context) {
            SecurityConstraint constraint = new SecurityConstraint();
            constraint.setUserConstraint("CONFIDENTIAL");
            SecurityCollection collection = new SecurityCollection();
            collection.addPattern("/*");
```

```

        constraint.addCollection(collection);
        context.addConstraint(constraint);
    }
};
tomcat.addAdditionalTomcatConnectors(httpToHttpsRedirectConnector());
return tomcat;
}

private Connector httpToHttpsRedirectConnector() {
    Connector connector = new Connector(TomcatServletWebServerFactory.DEFAULT_PROTOCOL);
    connector.setScheme("https");
    connector.setPort(12999);
    connector.setSecure(true);
    connector.setRedirectPort(12999); // same port, but https:// scheme
    return connector;
}

```

Reference: Spring Boot docs —

[How-to: Enable multiple connectors](https://docs.spring.io/spring-boot/how-to/webserver.html#howto.webserver.configure-ssl).

Step 5 — Update CORS Configuration

In the same `application.yml`, update the CORS origins from `http://` to `https://`:

```

ai.gebo.security:
  cors:
    allowedOrigins: https://gebo.example.com:12999,https://localhost:4200

```

Step 6 — Restart

```

docker compose down
docker compose up -d

```

The application is now reachable at `https://gebo.example.com:12999`.

2.2 Microservices Deployment

The microservices stack has **19 services** listening on ports 13000–13019. The **gateway** (port 13000) is the only browser-facing service. The recommended approach is **TLS termination at the gateway only**, with internal service-to-service communication staying on HTTP inside the Docker network.

Option A — TLS at the Gateway Only (Recommended)

Only the gateway needs a keystore. The other services remain on HTTP inside the internal Docker bridge network (`gebo-net`).

Step 1 — Generate the Keystore

Same as monolithic — use `keytool` to create a PKCS12 keystore.

Step 2 — Mount the Keystore for the Gateway

Edit `dockers/gebo.microservices/docker-compose.yml`:

```
gateway:
  image: geboai/gateway.gebo.ai:${GEBO_VERSION:-1.0.2.0-SNAPSHOT}
  restart: unless-stopped
  depends_on: [eureka]
  environment:
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka:13017/eureka/
    GEBO_SSL_KEYSTORE_PASSWORD: "your-strong-password-here"    # ← NEW
  ports:
    - "8443:13000"      # ← external HTTPS on 8443 → internal 13000
  volumes:
    - /home/gebo.ai/secrets/gebo-ssl.p12:/opt/gebo.ai/secrets/gebo-ssl.p12:ro # ← NEW
  networks: [gebo-net]
```

Step 3 — Add SSL Properties to the Gateway

The gateway's own `application.yml` is at

`gebo.apps.parent/gebo.microservices.apps.parent/gateway.gebo.ai/src/main/resources/application.yml`.

Add a `server.ssl` block:

```
server:
  port: 13000
  ssl:
    enabled: true
    key-store-type: PKCS12
    key-store: file:/opt/gebo.ai/secrets/gebo-ssl.p12
    key-store-password: ${GEBO_SSL_KEYSTORE_PASSWORD}
    key-alias: gebo-ssl
```

⚠ Do NOT put `server.ssl` settings into the shared config file

(`dockers/gebo.microservices/config/application.yml`). That file is mounted at

`/opt/gebo.ai/config` and loaded as an additional location by **all 19 services**.

Adding `server.ssl` there would apply TLS to every microservice and break

internal service-to-service communication, because Eureka registration and

`lb://` load-balanced calls would switch to HTTPS without corresponding

certificates on each service.

You must rebuild the gateway image after changing its baked-in `application.yml`:

```
cd gebo.apps.parent/gebo.microservices.apps.parent
mvn -pl gateway.gebo.ai -am package -DskipTests
docker compose -f dockers/gebo.microservices/docker-compose.yml up -d --build gateway
```

Step 4 — Update CORS

Update `allowedOrigins` in the shared config or gateway config:

```
ai.gebo.security:
  cors:
    allowedOrigins: https://gebo.example.com:8443,http://localhost:4200
```

Option B — Full mTLS / HTTPS Between All Services (Enterprise)

For high-security environments where every service-to-service call must be encrypted:

1. Generate a **CA certificate** and import it into a **truststore**.
2. Generate a **unique server certificate** (signed by the CA) for **each** of the 19 services.
3. Add `server.ssl.*` to **each service's own** `application.yml` (not the shared one).
4. Register each service in Eureka with `secure-virtual-host-name`:

```
eureka:
  instance:
    non-secure-port-enabled: false
    secure-port-enabled: true
    secure-port: ${server.port}
    status-page-url-path: ${server.servlet.context-path}/actuator/info
    health-check-url-path: ${server.servlet.context-path}/actuator/health
    home-page-url-path: ${server.servlet.context-path}/
```

5. Configure Spring Cloud Gateway routes with `lb:https://service-id` instead of `lb:http://service-id`.

This is a significant operational burden and is only recommended when a compliance framework (PCI-DSS, FedRAMP, etc.) mandates it.

Reference: Spring Cloud documentation —

[TLS/SSL in Spring Cloud](https://cloud.spring.io/spring-cloud-static/spring-cloud.html#_tls_and_ssl).

2.3 Certificate Generation Reference

Method	Command	Notes
Self-signed (keytool)	<code>`keytool -genkeypair -alias gebo-ssl -keyalg RSA -keysize 2048 -storetype PKCS12 -keystore gebo-ssl.p12 -validity 3650`</code>	Interactive; suitable for internal/staging use only
Self-signed (openssl)	<code>`openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 3650 -nodes && openssl pkcs12 -export -in cert.pem -inkey key.pem -out gebo-ssl.p12 -name gebo-ssl`</code>	Non-interactive; suitable for scripting
Let's Encrypt (Certbot)	<code>`certbot certonly --standalone -d gebo.example.com && openssl pkcs12 -export -in /etc/letsencrypt/live/gebo.example.com/fullchain.pem -inkey /etc/letsencrypt/live/gebo.example.com/privkey.pem -out gebo-ssl.p12 -name gebo-ssl`</code>	Requires a public domain name; cert auto-renews
Internal CA	Follow your organisation's PKI procedure; import the signed cert with <code>`keytool -importcert`</code>	Recommended for enterprise

For Let's Encrypt auto-renewal, set up a cron job that regenerates the PKCS12 file and copies it to `/home/gebo.ai/secrets/gebo-ssl.p12`, then signals the Docker container to load the updated keystore.

Reference: Spring Boot docs —

[How-to: Configure SSL](https://docs.spring.io/spring-boot/how-to/webserver.html#howto.webserver.configure-ssl).

3. Part B — Externalising the Encryption Keystore

3.1 Current State — Bundled Keystore

Gebo.ai ships with a **bundled PKCS12 keystore** inside the JAR at `/keystore/bundled.gebo.ai.ks`. This keystore contains an **AES** symmetric key used to encrypt/decrypt all sensitive secrets at rest in MongoDB — JWT signing secrets, OAuth2 client secrets, API keys for LLM providers, database passwords, and more.

Relevant source files:

File	Purpose
<code>`gebo.architecture.crypting/.../GeboCryptingServiceImpl.java:87–91`</code>	Loads the bundled keystore; hardcoded password <code>`TheFunkyHeadHunter1969Ad`</code>
<code>`gebo.architecture.crypting/.../KeyStoreConfig.java:21–39`</code>	External keystore configuration model
<code>`gebo.architecture.crypting/.../GeboCryptingConfig.java`</code>	Binds to <code>`ai.gebo.crypting.*`</code> properties

The initialisation logic (from `GeboCryptingServiceImpl.java:230–292`) is:

```
IF ai.gebo.crypting.keystoreConfig.keystore-path IS SET
  → Load the external keystore from that path
  → Read the key using configured alias and passwords
  → usedBundled = false
ELSE
  → Load /keystore/bundled.gebo.ai.ks from the classpath
  → Use hardcoded password and key alias
  → usedBundled = true
```

⚠ The bundled keystore password is hardcoded in Java source code and the keystore file is accessible to anyone who can extract the JAR.**

For any production deployment, you must configure an external keystore.

3.2 Generating a Custom Keystore

Use the Java `keytool` to generate a PKCS12 keystore with an AES key:

```
keytool -genseckey \
  -alias gebo-prod-key \
  -keyalg AES \
  -keysize 256 \
  -storetype PKCS12 \
  -keystore /home/gebo.ai/secrets/gebo-crypto.p12 \
  -storepass "your-keystore-password" \
  -keypass "your-key-password"
```

Note on AES key length: AES-256 requires the ****Java Cryptography Extension (JCE)**

Unlimited Strength Jurisdiction Policy** to be installed. For OpenJDK 11+, this is enabled by default. For older JDKs you may need to install the policy files.

Verify the keystore contents:

```
keytool -list -v -keystore /home/gebo.ai/secrets/gebo-crypto.p12 -storepass "your-keystore-password"
```

Expected output shows one `SecretKeyEntry` with alias `gebo-prod-key` and algorithm `AES`.

Generating with openssl (alternative):

```
# Generate a random 256-bit AES key
openssl rand -hex 32 > /tmp/aes-key.txt

# Create a PKCS12 keystore programmatically via a small Java tool:
# keytool cannot directly import a raw AES key from openssl.
# Use the keytool method above.
```

3.3 Configuring the External Keystore — Monolith

The external keystore is configured via the `ai.gebo.crypting.keystoreConfig.*` properties. These properties are **not** set in the shipped `application.yml` and must be **added** explicitly.

Step 1 — Mount the Keystore

Edit `dockers/gebo.ai/docker-compose.yml`:

```
gebo.ai:
  volumes:
    - /home/gebo.ai/secrets/gebo-crypto.p12:/opt/gebo.ai/secrets/gebo-crypto.p12:ro # ← NEW
```

Step 2 — Add the Configuration

Edit `dockers/gebo.ai/config/application.yml` and add the following block:

```
ai:
  gebo:
    crypting:
      keystoreConfig:
        key-store-type: PKCS12
        key-store-path: /opt/gebo.ai/secrets/gebo-crypto.p12
        key-algorithm: AES
        key-store-password: ${GEB0_CRYPT0_KEYSTORE_PASSWORD}
        key-password: ${GEB0_CRYPT0_KEY_PASSWORD}
        key-name: gebo-prod-key
```

Pass the passwords via environment variables:

```
gebo.ai:
  environment:
    GEB0_CRYPT0_KEYSTORE_PASSWORD: "your-keystore-password"
    GEB0_CRYPT0_KEY_PASSWORD: "your-key-password"
```

Step 3 — Restart and Verify

```
docker compose down && docker compose up -d
docker logs gebo.ai 2>&1 | grep -i "custom keystore"
```

Successful output:

```
Reading custom keystore security data structures
Security custom keystore data structures read!!
```

If the keystore cannot be loaded, the application will **fail to start** with a

`RuntimeException` — this is a deliberate fail-closed design.

Important: Once an external keystore is configured, any secrets previously encrypted with the bundled keystore will **no longer be decryptable**. You must re-encrypt all secrets with the new key. In practice this means:

- Re-entering LLM API keys in the admin UI
- Re-entering OAuth2 client secrets
- Re-entering any other stored credentials

>

Plan this as a fresh-start migration or schedule a maintenance window.

3.4 Configuring the External Keystore — Microservices

In the microservices architecture, **every service that encrypts/decrypts secrets** carries the `gebo.architecture.crypting` module and therefore needs access to the same keystore. This includes:

- `brain` (LLM API keys, prompt secrets)
- `vectorizator` (embedding provider keys)
- `gateway` (JWT secret verification)
- `heimdall` (the central secrets service — itself uses the keystore for storage)
- All content-handler services if they store credentials (SharePoint, Confluence, Jira, etc.)

Step 1 — Generate and Place the Keystore

```
keytool -genseckey -alias gebo-prod-key -keyalg AES -keysize 256 \
-storetype PKCS12 -keystore /home/gebo.ai/secrets/gebo-crypto.p12
```

Step 2 — Mount the Keystore into All Microservices

Add a volume mount to the shared `x-microservice`: anchor in

`dockers/gebo.microservices/docker-compose.yml`:

```
x-microservice: &microservice
restart: unless-stopped
depends_on: [eureka, rabbit, mongo]
environment:
  <<: *microservice-env
  GEBO_CRYPTO_KEYSTORE_PASSWORD: "your-keystore-password" # ← NEW (shared env)
  GEBO_CRYPTO_KEY_PASSWORD: "your-key-password" # ← NEW (shared env)
volumes:
  - ./config:/opt/gebo.ai/config:ro
  - /home/gebo.ai/secrets/gebo-crypto.p12:/opt/gebo.ai/secrets/gebo-crypto.p12:ro # ← NEW
  - /opt/gebo.ai/home
  - /opt/gebo.ai/work
networks: [gebo-net]
```

Step 3 — Add Keystore Config to the Shared Configuration

Edit `dockers/gebo.microservices/config/application.yml` and add:

```
ai:
  gebo:
    crypting:
      keystoreConfig:
        key-store-type: PKCS12
        key-store-path: /opt/gebo.ai/secrets/gebo-crypto.p12
        key-algorithm: AES
        key-store-password: ${GEBO_CRYPTOSTORE_PASSWORD}
        key-password: ${GEBO_CRYPTOSTORE_KEY_PASSWORD}
        key-name: gebo-prod-key
```

⚠ **This IS safe to put in the shared config** — unlike `server.ssl`, the keystore configuration is needed by every service that carries the crypting module (which is all services), and it is the same keystore everywhere.** The `gcloud` comment in the docker-compose about heimdall's secrets-cluster properties should NOT go here refers to heimdall-specific cluster endpoint settings, not to the shared crypto keystore.

Step 4 — Restart All Services

```
docker compose -f dockers/gebo.microservices/docker-compose.yml down
docker compose -f dockers/gebo.microservices/docker-compose.yml up -d
```

Key Rotation for Microservices

The same caveat applies as for the monolith: ****changing the keystore invalidates all previously encrypted secrets****. In a microservices deployment this is especially critical because:

- **heimdall** stores encrypted secrets in its own MongoDB database (`heimdall-gebo`). These secrets are served to other services as stored ciphertext; each caller decrypts locally. All callers must have the **same** keystore as heimdall.
- If you rotate the keystore, you must either:
 1. **Re-seed all secrets** through heimdall's admin API with the new key, or
 2. Accept a clean start with fresh secrets.

The `heimdall` service never decrypts on a caller's behalf — it hands back the **stored ciphertext** and the caller decrypts locally. No secret is ever in the clear on the network. This architecture is documented at `dockers/gebo.microservices/docker-compose.yml:173–176`.

4. Part C — Secrets Rotation Checklist

The following secrets are shipped with placeholder values and **must be rotated**

before production:

Secret	Configuration Property	How to Rotate
JWT signing secret	`ai.gebo.security.auth.tokenSecret`	Generate with `openssl rand -hex 64`; update in `application.yml`. All existing tokens are immediately invalidated — users must re-authenticate.
MongoDB root password	`MONGO_INITDB_ROOT_PASSWORD` (Docker env) + `ai.gebo.mongodb.connectionString`	Change docker-compose env, update connection string.
Qdrant API key	`QDRANT__SERVICE__API_KEY` (Docker env) + `ai.gebo.vectorstore.qdrant.apiKey`	Generate new UUID; update both places.
Neo4j password	`NEO4J_AUTH` (Docker env) + `spring.neo4j.authentication.password`	Change docker-compose env, update app config.
OpenSearch admin password	`OPENSEARCH_INITIAL_ADMIN_PASSWORD` (Docker env) + `ai.gebo.opensearch.password`	Change docker-compose env, update app config.
RabbitMQ password	`spring.rabbitmq.password` + `ai.gebo.messaging.rabbitmq.connection.password` (microservices)	Change in shared config and RabbitMQ config.
OAuth2 client secrets	`ai.gebo.security.oauth2configs[].client.secret`	Rotate in the OAuth2 provider's admin console, then update in Gebo.
Mail server credentials	`ai.gebo.userflows.mail-user-name`, `mail-password`	Update SMTP server credentials.
Encryption keystore password	`ai.gebo.crypting.keystoreConfig.key-store-password`	See [Section 3](#3-part-b--externalising-the-encryption-keystore).
TLS keystore password	`server.ssl.key-store-password`	See [Section 2](#2-part-a--enabling-https-sslts).

JWT Secret Rotation Procedure

```
# Generate a new 512-bit hex secret
NEW_SECRET=$(openssl rand -hex 64)
echo "New JWT secret: $NEW_SECRET"

# Update both monolith and microservices config files
# Replace the value of ai.gebo.security.auth.tokenSecret with $NEW_SECRET
```

Environment Variable Injection (Recommended)

Instead of hardcoding secrets in YAML files, use environment variable placeholders:

```
ai.gebo.security.auth:
  tokenSecret: ${GEB0_JWT_SECRET}
```

Set the values in a `.env` file (excluded from version control) or via your

container orchestration's secret management (Docker Secrets, Kubernetes Secrets,

HashiCorp Vault, etc.).

Reference: Spring Boot documentation —

[Externalized Configuration](https://docs.spring.io/spring-boot/reference/features/external-config.html).

5. Part D — Network & Infrastructure Hardening

5.1 Docker Network Binding

By default, the monolith compose file exposes infrastructure databases on

127.0.0.1 (loopback only), which is correct. Verify:

```
mongo:
  ports:
    - "127.0.0.1:27017:27017" # ✅ loopback only
qdrant:
  ports:
    - "127.0.0.1:6333:6333" # ✅ loopback only
```

In the microservices compose, infrastructure databases have **no ports published**

at all — they are reachable only within the internal Docker network `gebo-net`.

This is the most secure posture.

5.2 Microservices — Cluster Participant Restriction

The **heimdall** security service admits callers only from addresses currently

registered in Eureka. This is configured in heimdall's `application.yml`:

```
ai.gebo.cluster.participants:
  extra-service-ids: []
  additional-allowed-addresses: []
  cache-ttl: 30s
```

- **`extra-service-ids`** — Additional Eureka service IDs to admit. Keep empty.
- **`additional-allowed-addresses`** — Static IP bypasses. **Keep empty in production.**
- **`cache-ttl`** — How long a removed participant stays in the allow-list cache.

Also verify the **gateway** is NOT in the allow-list — see the comment at

`heimdall.gebo.ai/.../application.yml:53–63`. The gateway route only exposes

heimdall's admin API (`/heimdall/api/admin/**`), never its cluster secrets surface

(`/heimdall/api/cluster/**`).

5.3 OpenSearch — Internal SSL

OpenSearch already runs with internal SSL enabled:

```
- plugins.security.ssl.http.enabled=true
- plugins.security.ssl.transport.enabled=true
```

The `OPENSEARCH_SSL_VERIFICATIONMODE=none` setting in the OpenSearch Composer stack

is a **developer convenience** — consider enabling certificate verification in production.

5.4 Qdrant TLS

Qdrant currently has `tls: false`. To enable it in production:

1. Generate a TLS certificate for Qdrant.
2. Mount it and configure Qdrant's TLS settings.
3. Update `ai.gebo.vectorstore.qdrant.tls: true` in the application config.

5.5 Firewall Rules

Service	Port	Public Access	Recommendation
Monolith (HTTP)	12999	Yes (if hardened with HTTPS)	Allow only from trusted networks
Gateway (HTTP)	13000	Yes (if hardened with HTTPS)	Allow only from trusted networks
Eureka	13017	No	Block at firewall; internal only
All other microservices	13001–13019	No	Block at firewall; internal only
MongoDB	27017	No	Block at firewall
Qdrant	6333, 6334	No	Block at firewall
Neo4j	7474, 7687	No	Block at firewall
OpenSearch	9200	No	Block at firewall
RabbitMQ	5672, 15672	No	Block at firewall
Grafana	3000	No	Expose through authenticated reverse proxy
Prometheus	9090	No	Block at firewall